

University of Piraeus
Department of Informatics

Assignment Report

Set-Based Learning with Emergent Label Dynamics

Theoretical Foundations and Implementation Using Permutation-Invariant Models

In fulfilment of the requirements for the course
Advanced Deep Learning with PYTHON

Jan Bossenz
and
Annemarie Bianka Weise

Supervisor: Prof. Dr. Dionysios N. Sotiropoulos

Submission date: 2026-10-01

Contents

1	Introduction	1
2	Detailed theoretical analysis	2
2.1	Mathematical formulation of the synthetic problem	2
2.2	Deep Sets	2
2.3	Set Transformers	4
2.4	Janossy Pooling	6
2.5	PointNet++	8
3	Implementation and design details	13
3.1	Synthetic dataset creation	13
3.2	Dataset preparation	15
3.3	Model architectures	16
3.3.1	Deep Sets	16
3.3.2	Set Transformer	18
3.3.3	Janossy Pooling	22
3.3.4	PointNet++	24
3.4	Experiment execution	28
4	Experimental results	29
4.1	Overall and class-specific classification performance	29
4.2	CiteSeer classification performance by node degree	32
4.3	Computational efficiency	34
5	Discussion	36
5.1	Synthetic dataset	36
5.2	CiteSeer dataset	38
5.3	Practical implications and limitations	40
6	Tool usage reflection	41
7	References	42

1 Introduction

Machine learning comprises methods that learn statistical relationships from sample data and use them to classify new observations or make predictions. Advances in data availability, computational resources and learning algorithms have enabled these methods to be applied to increasingly complex problems. Machine learning is now used in numerous fields, ranging from commercial applications to scientific research. It is particularly relevant in disciplines in which large or structurally complex datasets must be analysed. One prominent example is bioinformatics. Genomic datasets often contain high-dimensional sequence and interaction information from which functional properties are to be derived. Machine learning methods are used, amongst other things, to analyse regulatory sequences, predict biological functions and evaluate genetic variants [4]. Alipanahi et al., for example, demonstrated with DeepBind that deep learning can learn the sequence specificity of DNA- and RNA-binding proteins from experimental data [1]. This makes it possible to predict the binding preferences of transcription factors and investigate the potential effects of sequence variations on protein binding. This example illustrates that reliable classification often depends not only on individual features, but also on their local or structural context.

Many of these learning problems cannot be formulated solely as processing of a single feature vector. Instead, the input consists of a set of related elements. Examples include the points in a point cloud, the interaction partners of a molecule and the neighbours of a node in a graph. There is no natural order for these sets. A change in the order in which their elements are stored or passed to a model should therefore not result in a different prediction. Models for set-based data must either ensure this permutation invariance directly through their architecture or achieve it through a suitable approximation [3, 5, 8, 7].

This report examines permutation-invariant models in the context of set-based node classification and emergent label dynamics. Four model families are compared with respect to their processing of this neighbourhood information. Deep Sets employs direct permutation-invariant sum aggregation, whilst the Set Transformer models relationships between set elements using attention. Janossy Pooling processes random permutations of the neighbourhood using a sequence-sensitive GRU and approximates permutation invariance through averaging. PointNet++, on the other hand, generates a hierarchical representation of the input set through sampling, grouping and local aggregation. The project presents the theoretical foundations of these approaches and compares their properties using standardised implementations. The evaluation is conducted on a synthetic dataset generated for this study and the CiteSeer citation network. It considers overall classification performance and class-specific differences. The computational costs of the models are also examined. A central question is whether more complex forms of neighbourhood processing provide a measurable advantage over simple aggregation under the investigated conditions. The associated training and inference costs are considered alongside any performance improvements.

2 Detailed theoretical analysis

2.1 Mathematical formulation of the synthetic problem

The synthetic classification problem represents a binary classification task in which the final label of a data point does not depend only on its own features, but rather on its local neighborhood, more precisely on the class distribution within its local neighborhood.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a dataset with n observations, where $x_i \in \mathbb{R}^D$ denotes the D -dimensional feature vector of the i -th observation and $y_i \in \{0, 1\}$ denotes its original binary class label. The feature vectors are generated from multivariate normal distributions depending on their class: $x_i \mid y_i = c \sim \mathcal{N}(\mu_c, \sigma^2 I_D)$, where $c \in \{0, 1\}$. The vectors $\mu_0, \mu_1 \in \mathbb{R}^D$ are the mean vectors of the two classes, σ^2 is the variance used within a class and I_D is the D -dimensional identity matrix. The distance between two data points is determined by the Euclidean distance. Thereby, the k nearest neighbors are defined for each data point. Using the original labels, the average neighborhood label is determined as $\bar{y}_i = \frac{1}{k} \sum_{x_j \in N_k(x_i)} y_j$. Due to the binary nature of the labels, $\bar{y}_i$ can be interpreted as the proportion of neighbors that belong to class 1. The final label is then obtained through the transformation $\hat{y}_i = 1 - y_i$ if $|\bar{y}_i - 0.5| < \varepsilon$ and $\hat{y}_i = y_i$ otherwise. The parameter ε determines the size of the interval around 0.5 that is considered ambiguous.

Since the neighborhood does not represent an ordered sequence, the order of the neighboring points must not affect the final prediction. Therefore, every permutation of the neighboring points should produce the same result. To satisfy this condition, permutation-invariant model architectures are required.

2.2 Deep Sets

DeepSets, as proposed by Zaheer et al. [8], are neural networks for set-based data. The input is mathematically represented as a set rather than as an ordered sequence. Therefore, the order of the input elements must not affect the result. Mathematically, if the set is

$$X = \{x_1, x_2, \dots, x_M\}, \quad x_m \in \mathcal{X},$$

then the function operating on this set has to be invariant under permutations [8]. This means that, for any permutation π ,

$$f(\{x_1, \dots, x_M\}) = f(\{x_{\pi(1)}, \dots, x_{\pi(M)}\}).$$

The problem with conventional neural networks operating on fixed-dimensional inputs is that their output generally depends on the positions of the input values. In a supervised environment in a set context, the result, meaning a classification or regression output, should remain unchanged when the order of the predictors is changed [8].

In an unsupervised environment, a score can indicate how well a set fits into the overall context; the paper describes such sets as “valid”. At the same time, in a transductive setting, individual predictions are made for every element of the input set. These predictions should remain associated with the corresponding elements when the input

elements are permuted [8]. For permutation-invariant set-level outputs, DeepSets use the following structure:

$$f(X) = \rho \left(\sum_{x \in X} \phi(x) \right). \quad (1)$$

Zaheer et al. [8] showed that according to the representation theorem from the paper, a permutation-invariant set function on a countable domain with suitable transformations ϕ and ρ can be brought into this specific form. More detailed proofs and derivations can be found in the original paper. The mappings are defined as

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^D \quad \text{and} \quad \rho : \mathbb{R}^D \rightarrow \mathcal{Y}.$$

The function ϕ maps each element $x_m \in \mathcal{X}$ to a D -dimensional representation. Since all representations $\phi(x_m)$ belong to $\mathbb{R}^D$, their sum also belongs to $\mathbb{R}^D$. Therefore, another requirement is fulfilled: the dimensionality of the aggregated form is not dependent on the size of the set M . Consequently, the same network ρ is able to process sets with varying numbers of elements. The function ρ then maps this aggregated representation to the desired output space $\mathcal{Y}$. Each element is first processed individually by ϕ . Thus,

$$h_m = \phi(x_m).$$

All elements x_m are processed by the same neural network ϕ using the same parameters. Afterwards, the sum of all representations $\phi(x_m)$ is computed. Since addition is commutative, a permutation changes only the order of the summands, but not the resulting sum. Therefore,

$$\sum_{m=1}^M \phi(x_{\pi(m)}) = \sum_{m=1}^M \phi(x_m).$$

The resulting architecture is consequently permutation invariant [8]. The function ϕ is applied before the summation in order to preserve relevant information about the individual elements. This information can, for example, be represented by transforming each element into a feature space of possibly higher dimension. Thereby, the element-wise application of ϕ to every element of the set with identical parameters,

$$\Phi(X) = (\phi(x_1), \dots, \phi(x_M)),$$

also results in this mapping being permutation equivariant. For any permutation π ,

$$\Phi(\pi X) = (\phi(x_{\pi(1)}), \dots, \phi(x_{\pi(M)})) = \pi \Phi(X).$$

This allows its application in a transductive setting.

The function ρ is then applied to the resulting sum in order to obtain the desired output. In supervised learning, this output could be a class probability or a regression value, while in unsupervised learning it could be a score for the entire set [8]. One limitation of Deepsets is that by processing each element individually and then aggregating

the outcomens, interactions between the elements are not sufficiently modeled.

2.3 Set Transformers

As already described in the previous section, the interactions between the individual elements transformed by ϕ may not be sufficiently modelled by DeepSets. Set Transformers attempt to solve this problem. For this purpose, they build on the fundamental structure of DeepSets and the mathematical proofs already presented. Thus, the fundamental structure of DeepSets is used:

$$f(X) = \rho(\text{pool}\{\phi(x_1), \dots, \phi(x_M)\}) \quad [8, 3].$$

This is then conceptually divided into an encoder, the function ϕ , which processes the individual elements and the decoder, which consists of the pooling operation and the subsequently applied function ρ . Since ϕ is only applied to each element individually and only a fixed pooling operation is performed afterwards, interactions between individual elements may not be sufficiently modelled [3]. Set Transformers use attention to solve this problem. Based on given matrices of queries Q , keys K , and values V , attention is defined as

$$\text{Att}(Q, K, V; \omega) = \omega(QK^\top)V \quad [3].$$

Here, $QK^\top$ contains the pairwise dot products between queries and keys, which are used as similarity scores. The function ω transforms these scores into attention weights. The final output $\omega(QK^\top)V$ is a weighted sum of the values, where values whose keys have a higher similarity to the corresponding query are weighted more strongly. This mechanism is then extended by multihead attention, in which several attention operations are performed simultaneously. For this purpose, the queries, keys and values are projected into different representation spaces for each attention head. Afterwards, the outputs of the individual heads are concatenated and transformed:

$$\text{Multihead}(Q, K, V; \lambda, \omega) = \text{concat}(O_1, \dots, O_h)W^O, \quad O_j = \text{Att}(QW_j^Q, KW_j^K, VW_j^V; \omega_j) \quad [3].$$

In this representation, the matrices W_j^Q , W_j^K and W_j^V contain learnable parameters. Since each attention head uses different projections, different interactions between the elements can also be considered. The outputs of the individual heads are then combined using the matrix W^O [3].

Based on this multihead attention mechanism, the Set Transformer introduces the Multihead Attention Block, abbreviated as MAB. Given the two matrices $X \in \mathbb{R}^{M \times D}$ and $Y \in \mathbb{R}^{N \times D}$, the block is defined as

$$\text{MAB}(X, Y) = \text{LayerNorm}(H + \text{rFF}(H)), \quad H = \text{LayerNorm}(X + \text{Multihead}(X, Y, Y)) \quad [3].$$

Here, rFF denotes a row-wise feedforward network. In this context, the same feedforward network with the same parameters is applied independently to each row. In addition, the MAB contains residual connections and layer normalization. Using this MAB, a Set Attention Block, abbreviated as SAB, is defined. It applies the MAB to the

same matrix and therefore computes self-attention over the set, resulting in another set of the same size. Formally, $\text{SAB}(X) = \text{MAB}(X, X)$ [3]. In this way, each element of the set can interact with every other element, which distinguishes this mechanism from the element-wise transformation in DeepSets. By stacking several SABs, higher-order interactions can also be captured [3]. Since each element is compared with every other element during self-attention, the operation has a computational complexity of $\mathcal{O}(M^2)$, which can be computationally expensive for large input sets [3].

To address this problem, the Set Transformer also introduces the Induced Set Attention Block, abbreviated as ISAB. For this purpose, m D -dimensional trainable inducing points $I \in \mathbb{R}^{m \times D}$ are introduced. An ISAB is defined as

$$H = \text{MAB}(I, X), \quad \text{ISAB}_m(X) = \text{MAB}(X, H) \quad [3].$$

The advantage is that, instead of directly comparing the M elements with each other, the intermediate representation $H \in \mathbb{R}^{m \times D}$ is first created by using the m inducing points as queries and the M input elements as keys and values. In the second MAB, the M input elements are used as queries, while the m elements of H are used as keys and values. This results in $\text{MAB}(X, H) \in \mathbb{R}^{M \times D}$, which has the original number of elements. Therefore, the computational complexity is $\mathcal{O}(Mm)$. In general, the number of inducing points is considerably smaller than the total number of elements in the set, which makes the ISAB more efficient than the SAB. Similar to the SAB, the ISAB produces a set with the same number of elements as its input and is permutation equivariant, meaning that $\text{ISAB}_m(\pi X) = \pi \text{ISAB}_m(X)$ [3].

After the input has been processed by SABs or ISABs, meaning that the encoder step of the model has been completed, these element-wise representations still have to be aggregated. For this purpose, the Set Transformer uses Pooling by Multihead Attention, abbreviated as PMA. Using the set $Z \in \mathbb{R}^{M \times D}$ produced by the encoder and a learnable set of k seed vectors $S \in \mathbb{R}^{k \times D}$, PMA is defined as

$$\text{PMA}_k(Z) = \text{MAB}(S, \text{rFF}(Z)) \in \mathbb{R}^{k \times D} \quad [3].$$

The seed vectors act as queries, while the set produced by the encoder provides the keys and values. This architecture enables the model to learn how strongly each element should influence each of the k output vectors. In Lee et al. [3], a single seed vector is used in most cases, while some problems may require multiple seed vectors. Since the resulting output vectors may depend on each other, an additional SAB can be applied after PMA:

$$H = \text{SAB}(\text{PMA}_k(Z)) \quad [3].$$

This allows interactions between the resulting output vectors to be modelled.

The complete Set Transformer architecture consists of an encoder followed by a decoder, where the encoder is formed by stacking SABs or ISABs. In this way, the number of elements in the set is preserved throughout the encoding process. The decoder then aggregates the output of the encoder using PMA and processes the resulting

vectors using a row-wise feedforward network. An SAB following PMA is particularly relevant when $k > 1$, as it allows dependencies between the output vectors to be modelled.

Since SAB and ISAB are permutation equivariant, the complete encoder satisfies

$$\text{Encoder}(\pi X) = \pi \text{Encoder}(X) \quad [3].$$

Similarly, PMA is permutation invariant with respect to the encoded representations:

$$\text{PMA}_k(\pi Z) = \text{PMA}_k(Z) \quad [3].$$

Consequently, the complete Set Transformer architecture is permutation invariant.

2.4 Janossy Pooling

Janossy Pooling is another approach for ensuring permutation invariance when transforming sets. It differs from the previously described approaches in that the function applied to the input does not need to be permutation invariant. Instead, it allows the use of permutation-sensitive functions and achieves permutation invariance by averaging the outputs obtained from all different input orderings.

Let $X = (x_1, \dots, x_M)$ be a finite input sequence representing a set or multiset. Π_M denotes the set of all permutations of the indices $\{1, \dots, M\}$. For a permutation $\pi \in \Pi_M$, the reordered input is represented as $X_\pi = (x_{\pi(1)}, \dots, x_{\pi(M)})$. Janossy Pooling then defines the permutation-invariant function

$$f_J(M, X; \theta^{(f)}) = \frac{1}{M!} \sum_{\pi \in \Pi_M} f^*(M, X_\pi; \theta^{(f)}) \quad [5].$$

Here, f^* represents a potentially permutation-sensitive function. It can, for example, be implemented using a feedforward network, an RNN, an LSTM, a GRU, or another model that can process variable-length sequences. Although f^* itself is not necessarily permutation invariant, averaging over all input orderings makes the result permutation invariant. Since the sum contains every possible permutation of the input, the initial ordering of the elements has no influence on the result.

If the function f^* is already powerful enough to model all relationships required for the task, the resulting Janossy function can theoretically represent any permutation-invariant function. In this case, ρ is not necessarily required as an additional function. In practice, however, ρ can improve the quality of the model by extracting relevant information from the averaged output. It can also partly compensate for information that is lost when an approximate or restricted form of Janossy Pooling is used [5]. The main disadvantage of Janossy Pooling is its computational cost. For each set containing M elements, the function f^* must be applied to every possible permutation and therefore has to be evaluated $M!$ times. The paper describes three strategies for keeping this computational cost manageable.

The first strategy uses a canonical ordering of the input. The canonical ordering function maps all permutations of the same input to the same ordered sequence. It therefore satisfies $\text{CANONICAL}(X) = \text{CANONICAL}(X_\pi)$ for every $\pi \in \Pi_M$. The permutation-sensitive function then only has to be applied once to the output of the canonical ordering:

$$f_{\text{canonical}}(X) = f^*(M, \text{CANONICAL}(X)) \quad [5].$$

This approach requires a meaningful canonical ordering. If the selected ordering is not suitable for the prediction task, important relationships in the data may be difficult to learn. It is also possible to attempt to learn the appropriate ordering function itself. However, this is also a computationally expensive optimization problem because the search space again contains $M!$ possible permutations [5].

The second strategy is k -ary Janossy Pooling. In this case, f^* does not depend on the entire reordered input, but only on the first k elements of each permutation. Let $\downarrow_k(X_\pi)$ denote the projection of X_π onto its first k elements. The corresponding Janossy function is defined as

$$f_{J,k}(M, X) = \frac{1}{M!} \sum_{\pi \in \Pi_M} f^*(M, \downarrow_k(X_\pi)) \quad [5].$$

If $M < k$, the remaining positions of the sequence are filled using zero padding. This restriction assumes that the relationships captured by the first k elements are sufficient for the respective task. Many permutations have the same first k elements and therefore produce the same output. As a result, only $\frac{M!}{(M-k)!}$ distinct ordered selections of k elements have to be evaluated. The operation can therefore also be written as

$$f_{J,k}(M, X) = \frac{(M-k)!}{M!} \sum_{(i_1, \dots, i_k) \in \mathcal{I}_M} f^*(M, (x_{i_1}, \dots, x_{i_k})) \quad [5],$$

where $\mathcal{I}_M$ denotes all ordered selections of k distinct indices from $\{1, \dots, M\}$. For small values of k , the required computational effort is substantially lower than for exact Janossy Pooling. However, k represents a trade-off between computational cost and model capacity. A connection to DeepSets can also be established: if $k = 1$ is selected together with suitable functions f^* and ρ , the resulting model can be interpreted as a DeepSet.

The third strategy is permutation sampling. Instead of considering every possible permutation, a permutation s is sampled uniformly from Π_M . The complete Janossy average is then replaced by

$$\hat{f}_J(M, X) = f^*(M, X_s), \quad s \sim \text{Unif}(\Pi_M), \quad \mathbb{E}_s [\hat{f}_J(M, X)] = f_J(M, X) \quad [5].$$

The representation obtained from a single sampled permutation is therefore an unbiased estimator of the exact Janossy representation. This also allows complex permutation-sensitive models, such as LSTMs or GRUs, to be used without evaluating all $M!$ permutations. During training, a new random permutation of each input sequence in a mini-batch is sampled. The authors refer to this optimization procedure as π -SGD. In simplified form, it

optimizes the objective

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{s_i} \left[\mathcal{L} \left(y^{(i)}, \rho \left(f^* \left(M_i, X_{s_i}^{(i)} \right) \right) \right) \right] \quad [5].$$

Although $\hat{f}_J$ is an unbiased estimator of the exact Janossy representation, this does not necessarily apply to the complete prediction or the resulting loss. This is because ρ and $\mathcal{L}$ are nonlinear in most cases. Consequently, averaging the representations before applying these functions does not necessarily produce the same output as applying the functions first and averaging their outputs afterwards. It therefore follows that π -SGD does not usually optimize the exact objective of complete Janossy Pooling. Instead, it optimizes the tractable objective J . However, repeatedly presenting the same input to the model in different random orderings encourages the model to produce similar results for these orderings in order to minimize the loss. As a result, the model becomes less sensitive to the ordering of the input elements. Under conditions similar to those required for standard SGD, π -SGD also has corresponding convergence properties [5].

After training, the model may still depend on the particular permutation of the input. To reduce this dependence, the outputs of several independent permutations can be calculated and subsequently averaged:

$$\hat{y}_R(X) = \frac{1}{R} \sum_{r=1}^R \rho(f^*(M, X_{s_r})), \quad s_r \sim \text{Unif}(\Pi_M) \quad [5].$$

Here, R denotes the number of sampled permutations. The choice of R again represents a trade-off. For a small value of R , the prediction may strongly depend on the selected orderings. Increasing R reduces these fluctuations, but also increases the computational cost. This permutation-sampling approach can also be combined with k -ary Janossy Pooling.

2.5 PointNet++

PointNet++, proposed by Qi et al. [7], is the successor architecture to PointNet. PointNet processes every input point individually using the same point-wise transformation, while information from different points is combined only during the global max-pooling operation. Consequently, local spatial relationships between points can only be represented to a limited extent. PointNet++ addresses this limitation by organizing the input points hierarchically into local regions and progressively extracting features at increasingly larger scales.

A central assumption of PointNet++ is that the input points are sampled from a metric space

$$(\mathcal{M}, d), \quad \mathcal{M} \subseteq \mathbb{R}^D,$$

where

$$d : \mathcal{M} \times \mathcal{M} \rightarrow \mathbb{R}_{\geq 0}$$

denotes a distance function defined between pairs of points. The value D denotes the dimensionality of the coordinates of each point. The density of the sampled points does not necessarily have to be uniform. This is particularly

relevant for point clouds obtained from physical sensors, where some spatial regions may be sampled more densely than others. A point set containing N points can be represented as

$$P = \{(x_i, a_i)\}_{i=1}^N, \quad x_i \in \mathbb{R}^D, \quad a_i \in \mathbb{R}^C.$$

Here, x_i contains the coordinates of the i -th point, whereas a_i denotes its corresponding point features. Although P represents an unordered set, it can be stored computationally as a matrix

$$P \in \mathbb{R}^{N \times (D+C)}.$$

The objective is to learn a function that extracts semantic information from a complete set of points. Let $\mathcal{S}$ denote the space of finite point sets of the previously defined form. For a classification task, the model maps the complete point set to one output class:

$$F_{\text{class}} : \mathcal{S} \rightarrow \mathcal{Y}.$$

Since the input represents an unordered set, the classification result must not depend on the order in which the points are provided. For a permutation π of the N input points, let

$$P_\pi = \{(x_{\pi(i)}, a_{\pi(i)})\}_{i=1}^N$$

denote the corresponding reordered representation of the point set. The classification function must then satisfy

$$F_{\text{class}}(P_\pi) = F_{\text{class}}(P).$$

To extract both local and global information while satisfying these symmetry requirements, PointNet++ applies multiple hierarchical set-abstraction levels. Each set-abstraction level consists of a sampling layer, a grouping layer and a local PointNet layer. The resulting point set contains fewer points, while each remaining point is assigned a learned feature vector summarizing its local region. By repeatedly applying this operation, the network progressively combines information from increasingly larger regions. In the classification path, the remaining set is finally transformed into one global feature vector by a global set-abstraction operation. A fully connected classification head maps this vector to a set of class logits, from which one class for the complete input point set is derived. In contrast, the segmentation path propagates the abstracted features back to the original points to produce one prediction for each input point.

The sampling layer is responsible for selecting a subset of points that will serve as the centroids of local regions. For this purpose, PointNet++ uses iterative farthest point sampling (FPS). After an initial centroid has been selected, each subsequent centroid is chosen as the point whose distance to the set of previously selected centroids is maximal. Let

$$S_{j-1} = \{x_{i_1}, x_{i_2}, \dots, x_{i_{j-1}}\}$$

denote the set of centroids selected during the previous iterations. The index of the next centroid is determined by

$$i_j = \arg \max_{i: x_i \notin S_{j-1}} \min_{x_\ell \in S_{j-1}} d(x_i, x_\ell).$$

For each candidate point x_i , its distance to the closest previously selected centroid is calculated. The point for which this minimum distance is largest is selected as the next centroid. In this way, FPS distributes the centroids across the point set and generally provides better coverage than random sampling [7]. After N' iterations, the selected centroid coordinates can be represented by the matrix

$$C_{\text{centroid}} = \begin{pmatrix} c_1^\top \\ c_2^\top \\ \vdots \\ c_{N'}^\top \end{pmatrix} \in \mathbb{R}^{N' \times D}.$$

Here, N' denotes the number of selected centroids. Based on these selected centroids, the grouping layer subsequently constructs the corresponding local regions.

The grouping layer receives the current input point set P together with the coordinates of the N' centroids selected by the sampling layer. For each centroid c_j , a local neighborhood is constructed based on the metric distance between the centroid and the input points. One possible grouping method is K -nearest-neighbor search. For every centroid c_j , the indices of its K nearest input points are determined by

$$G_j = \arg \text{topK}_{\substack{\text{smallest} \\ i \in \{1, \dots, N\}}} d(x_i, c_j).$$

Here, $d(x_i, c_j)$ denotes the distance between the i -th input point and centroid c_j , according to the metric defined on the input space. A common choice is the Euclidean distance,

$$d(x_i, c_j) = \|x_i - c_j\|.$$

The resulting index set

$$G_j = (G_{j,1}, \dots, G_{j,K})$$

contains the indices of the K points with the smallest distances to centroid c_j . Since the centroids are selected from the input points, a centroid is generally also contained in its own neighborhood because $d(c_j, c_j) = 0$. Furthermore, the resulting local regions may overlap, meaning that the same input point can occur in the neighborhoods of multiple centroids. Combining the index sets of all N' centroids produces an index matrix

$$G \in \{1, \dots, N\}^{N' \times K}.$$

The corresponding coordinates and point features can then be collected from the current input point set, resulting

in a grouped tensor of size

$$N' \times K \times (D + C).$$

Each of the N' groups therefore represents one local region and contains K points with D -dimensional coordinates and C -dimensional point features. As an alternative to K -nearest-neighbor grouping, the original PointNet++ architecture also introduces radius-based ball queries. For a radius r , the neighborhood of centroid c_j is defined as

$$G_j^{(r)} = \{i \mid d(x_i, c_j) \leq r\}.$$

A ball query maintains a fixed metric scale, while the number of points within the region may vary depending on the local point density. In contrast, K -nearest-neighbor grouping maintains a fixed number of points, while the metric area covered by a group may vary [7].

The N' local regions constructed by the grouping layer are subsequently passed to the local PointNet layer. This layer is based on the fundamental local feature extraction operation of PointNet. Its purpose is to transform every local region, which may contain multiple points, into one fixed-dimensional feature vector associated with the corresponding centroid. First, the coordinates of the points within each local region are translated into a local coordinate frame relative to the corresponding centroid. For the k -th point in the neighborhood of centroid c_j , the relative coordinates are given by

$$r_{j,k} = x_{G_{j,k}} - c_j.$$

These relative coordinates are subsequently concatenated with the point features:

$$z_{j,k} = r_{j,k} \parallel a_{G_{j,k}},$$

where $\parallel$ denotes concatenation. The resulting vector $z_{j,k} \in \mathbb{R}^{D+C}$ therefore contains both the relative position of the point within the local region and its associated point features. Each vector $z_{j,k}$ is then processed by the same point-wise function h . This function is generally implemented as a shared multilayer perceptron (shared MLP) and can be written as

$$h : \mathbb{R}^{D+C} \rightarrow \mathbb{R}^H.$$

The use of the same function and the same parameters for every point ensures that the individual transformation does not depend on the position of a point in the input representation. The transformed point representations are subsequently aggregated using component-wise max pooling. For the local region associated with centroid c_j , the resulting feature vector is

$$a_j^{(s)} = \gamma_s \left(\text{MAX}_{i \in G_j^{(s)}} \{h_s((x_i - c_j) \parallel a_i)\} \right).$$

Here, $\gamma : \mathbb{R}^H \rightarrow \mathbb{R}^{C'}$ denotes an optional transformation applied after the max-pooling operation. It may, for example, be implemented as another MLP. If no additional transformation is required, γ can also be chosen as the identity function. In this case, $H = C'$. The max-pooling operation is performed independently for every feature

dimension. It therefore selects the strongest activation of each learned feature across all points of the local region. Since the maximum does not depend on the order of the points, the resulting local feature vector $a'_j \in \mathbb{R}^{C'}$ is permutation invariant with respect to the points within the region. The output of the complete set-abstraction level can consequently be represented as

$$P' = \{(c_j, a'_j)\}_{j=1}^{N'}, \quad P' \in \mathbb{R}^{N' \times (D+C')}.$$

Compared with the input point set, the number of points has been reduced from N to N' , while every remaining centroid is now associated with a feature vector summarizing its local neighborhood. This new point set can then be used as the input to the next set-abstraction level.

A challenge considered by PointNet++ is that the density of a point set may vary across different regions. A small local region may provide detailed structural information in a densely sampled area, while the same region may contain too few points to produce a reliable representation in a sparsely sampled area. PointNet++ therefore introduces different variants for extracting and combining features at multiple scales [7]. The simplest variant is single-scale grouping, in which one neighborhood is constructed for every centroid and processed by one local PointNet. In contrast, multi-scale grouping (MSG) constructs multiple neighborhoods with different metric scales around the same centroid. Using radius-based grouping, these neighborhoods can be written as

$$G_j^{(s)} = \{i \mid d(x_i, c_j) \leq r_s\}, \quad s = 1, \dots, S.$$

r_s denotes the radius of the s -th scale. Each neighborhood is processed by a separate local PointNet:

$$a_j^{(s)} = \gamma_s \left(\text{MAX}_{k \in G_j^{(s)}} \{h_s(z_{j,k})\} \right).$$

The resulting feature vectors are then concatenated:

$$a'_j = a_j^{(1)} \parallel a_j^{(2)} \parallel \dots \parallel a_j^{(S)}.$$

In this way, the model can combine detailed features extracted from small neighborhoods with more general features extracted from larger neighborhoods. To make the model more robust to different sampling densities, the PointNet++ paper additionally applies random input dropout during training. This exposes the network to point sets with varying densities and encourages it to learn how the information obtained from the different scales should be combined. However, MSG is computationally expensive because multiple grouping operations and local PointNets have to be evaluated for every centroid.

Multi-resolution grouping (MRG) provides a more computationally efficient alternative. Instead of processing several neighborhood scales independently at every level, MRG combines information obtained through two different paths. The first path uses the hierarchically abstracted features from the preceding level. These features summarize

smaller subregions and therefore preserve fine-grained local information. The second path directly processes the less abstracted points within a larger local region using a separate PointNet. The two resulting representations are concatenated:

$$a'_j = a_{j,\text{hierarchical}} \parallel a_{j,\text{direct}}.$$

The usefulness of these approaches depends on the local sampling density. In a densely sampled region, the hierarchical representation can preserve fine structural details because the lower-level subregions contain enough points. In a sparsely sampled region, these subregions may contain too few points and their representations may become unreliable. In this case, directly processing the points from the larger region can provide a more robust representation. By combining both feature vectors, subsequent layers can learn which information is more relevant for the respective local region. MSG and MRG therefore represent density-adaptive variants of the grouping and local PointNet operations.

3 Implementation and design details

3.1 Synthetic dataset creation

In the assignment, most of the parameter values required to generate the synthetic dataset are left unspecified. Without concrete values for N , D , σ , p , k and ϵ , it is difficult to determine how challenging the resulting classification problem should be, since its difficulty strongly depends on these parameters. To provide the models with classification problems of approximately similar difficulty on the synthetic and CiteSeer datasets, the synthetic dataset therefore had to be calibrated. For this purpose, the predictive performance of a simple reference classifier was used as an approximate measure of classification difficulty. In addition, a separation parameter Δ , named **distance** in the implementation, was introduced to control the overlap between the two Gaussian classes.

The initial data are generated by first assigning an original binary label to each of the N points:

$$y_i \sim \text{Bernoulli}(p).$$

To obtain approximately equally sized original classes, the class probability was fixed to $p = 0.5$. Depending on this label, each point is then sampled from one of two Gaussian distributions. Their D -dimensional mean vectors are positioned symmetrically around the origin:

$$\mu_0 = \left(-\frac{\Delta}{2}, \dots, -\frac{\Delta}{2}\right), \quad \mu_1 = \left(\frac{\Delta}{2}, \dots, \frac{\Delta}{2}\right).$$

The parameter Δ controls the coordinate-wise separation of the two class means, while σ determines the standard deviation within each class. In general, increasing the ratio between Δ and σ reduces the overlap between the two distributions. However, Δ is not directly equal to the Euclidean distance between the mean vectors. Since the mean

values differ by Δ in every feature dimension, their Euclidean distance is

$$\|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0\|_2 = \sqrt{D} \Delta.$$

Using these mean vectors, the feature vectors were sampled according to the class-conditional Gaussian formulation introduced in section 2.1. The neighborhood-dependent target labels were then constructed using the k nearest neighbors in the original feature space, excluding the point itself. All neighborhood proportions were calculated from the original labels, meaning that the label modifications were applied simultaneously rather than iteratively. The modified labels $\hat{y}_i$ were subsequently used as prediction targets for the models, while the original labels were retained for the analysis of the generated dataset.

Testing a large number of values for every unspecified parameter would have considerably increased the computational cost of the parameter search. Therefore, some parameters were fixed in advance. The number of points was set to $N = 3327$, corresponding to the number of nodes in CiteSeer. The feature dimension was fixed to $D = 8$ and the probability of the original class 1 was fixed to $p = 0.5$. To reduce the dependence of the parameter evaluation on one particular random generation, every parameter combination was tested using the seeds 0, 1 and 2. The following values were varied:

$$\begin{aligned}\sigma &\in \{0.5, 1.0, 1.5\}, \\ \Delta &\in \{1.0, 2.0, 3.0\}, \\ k &\in \{3, 5, 10\}, \\ \epsilon &\in \{0.05, 0.11, 0.21\}.\end{aligned}$$

This results in $3^4 = 81$ different parameter combinations. Since each combination was evaluated with three seeds, the complete search required $81 \cdot 3 = 243$ synthetic classification runs.

To estimate the difficulty associated with each parameter combination, a simple reference classifier was used. For every point i , the mean feature vector of its neighborhood was first calculated as

$$\bar{\mathbf{x}}_{\mathcal{N}(i)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{x}_j.$$

This neighborhood representation was then concatenated with the feature vector of the point itself:

$$\mathbf{z}_i = \mathbf{x}_i \parallel \bar{\mathbf{x}}_{\mathcal{N}(i)}.$$

The resulting vectors were standardized using a **StandardScaler** and classified using logistic regression. The classifier was configured with balanced class weights, the **lbfgs** solver and a maximum of 2000 iterations. The same construction was applied to CiteSeer, where the mean was calculated from the features of the nodes connected to the respective node in the undirected citation graph.

For every run, a stratified split containing 80% training data, 10% validation data and 10% test data was generated. Accuracy and Macro-F1 were calculated as evaluation metrics. Only the validation results were considered during the parameter selection, while the test results were retained as an independent final comparison. For each synthetic parameter combination, the results were averaged across the three generation seeds. Similar validation performance alone was not considered sufficient for selecting the final dataset. Parameter combinations were excluded if their average proportion of modified labels was below 10% or above 30%, since such configurations would result in either an almost unchanged dataset or an excessively large modification of the original labels. Furthermore, the proportion of the modified class 1 was required to remain between 35% and 65% to avoid a strongly imbalanced target distribution.

The remaining configurations were used to identify a suitable compromise between baseline performance, neighborhood size, label-flip rate and class balance. A neighborhood size of $k = 10$ was preferred over very small neighborhoods to obtain a more stable representation of the local class distribution. Among the remaining configurations with this neighborhood size, the raw validation accuracy and Macro-F1 were compared with the corresponding CiteSeer results. Based on this comparison and the additional requirements concerning label-flip rate and class balance, the final configuration was selected manually.

$$N = 3327, \quad D = 8, \quad \sigma = 1.5, \quad \Delta = 1.0, \quad p = 0.5, \quad k = 10, \quad \epsilon = 0.11.$$

After selecting these parameters, the final dataset used for training and evaluating the four models was generated separately with seed 42. The reference-classifier procedure provides an approximate empirical comparison of dataset difficulty rather than an exact equivalence between Synthetic and CiteSeer, since the two datasets differ in their number of classes, feature representations and neighborhood structures.

3.2 Dataset preparation

Although the synthetic and CiteSeer datasets are constructed differently, all four models require the same fundamental input structure. For every data point or node i , the models receive its feature vector $\mathbf{x}_i$ together with the unordered set containing the features of its neighbors:

$$X_{\mathcal{N}(i)} = \{\mathbf{x}_j \mid j \in \mathcal{N}(i)\}.$$

The neighborhoods are constructed once during dataset preparation and stored using the indices of the corresponding elements. Consequently, all four models start with the same feature matrix, target labels and neighborhood definitions and differ only in how they process the resulting neighborhood sets. For the synthetic dataset, the Euclidean k -nearest-neighbor sets described in the previous section are used. The features, original and modified labels and neighborhood indices are stored in `synthetic_dataset.pt`. During training and evaluation, the modified labels are used as prediction targets.

The CiteSeer dataset represents a citation network in which each node corresponds to a scientific publication. The node features describe its textual content, while the edges represent citation relationships between publications. The

graph is loaded using the `Planetoid` interface provided by PyTorch Geometric and converted into an undirected graph. The neighborhood of node i is therefore defined as

$$\mathcal{N}(i) = \{j \mid (i,j) \in E_{\text{undirected}}, j \neq i\}.$$

Self-loops and duplicated neighbors are removed. In contrast to the synthetic dataset, the number of neighbors depends on the degree of the respective node and can therefore vary between nodes. The node features, class labels, edge index and neighborhood indices are stored in `citeseer_dataset.pt`.

Both prepared dataset files are loaded through the same dataset-loading function. This function selects the modified labels for the synthetic dataset and the original class labels for CiteSeer. Apart from this distinction, all models receive the same feature matrix and neighborhood indices for a given dataset.

3.3 Model architectures

3.3.1 Deep Sets

Our implementation naturally follows the permutation-invariant formulation introduced in Eq. (1), adapted to node classification. The complete architecture is illustrated in Fig. 1. For each node i , an input set is defined based on its neighborhood $\mathcal{N}(i)$. This node-level mapping is implemented as

$$\mathbf{o}_i = \rho \left(\mathbf{x}_i \parallel \sum_{j \in \mathcal{N}(i)} \phi(\mathbf{x}_j) \right), \quad \mathbf{o}_i \in \mathbb{R}^{n_{\text{class}}}, \quad (2)$$

where $\mathbf{o}_i$ denotes the vector of unnormalized class scores for node i . The neighborhood is processed as a set, while the information of the current node is still included in the prediction.

The input to the model consists of a feature matrix containing the features of all nodes and a list containing the neighborhood indices of each node. In addition, the model receives the indices of the nodes for which predictions should be generated. The input dimension d_{node} corresponds to the number of features of each node, whereas the output dimension corresponds to the number of classes.

In our implementation, ϕ is represented by

$$\text{Linear}(d_{\text{node}}, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, 128) \rightarrow \text{ReLU}.$$

The dimension of the internal feature vectors was set to 128. Since all implemented models contain a parameter corresponding to `internal_features` or `dim_hidden`, we decided to use the same value in order to increase comparability. The GitHub implementation associated with the Set Transformer [3] uses a default hidden dimension of 128 for both its Deep Sets pipeline and its Set Transformer models.¹ The choice of the hidden dimension naturally represents a trade-off. A smaller value for the internal feature dimension can make the model more compact and

¹https://github.com/juho-lee/set_transformer/blob/master/models.py

reduce its memory requirements and computational cost. It may also reduce overfitting because the model has less capacity to memorize complex relationships. A larger hidden dimension increases the representational capacity of the model. This allows it to capture more complex structures, but it can also cause the model to adapt more strongly to the training data and its noise. Therefore, the risk of overfitting may increase, particularly for small datasets.

The representations calculated by ϕ are summed afterwards. Instead of the sum, other permutation-invariant functions could have been used at this point. We selected summation because it is also the central aggregation function in the original Deep Sets paper [8].

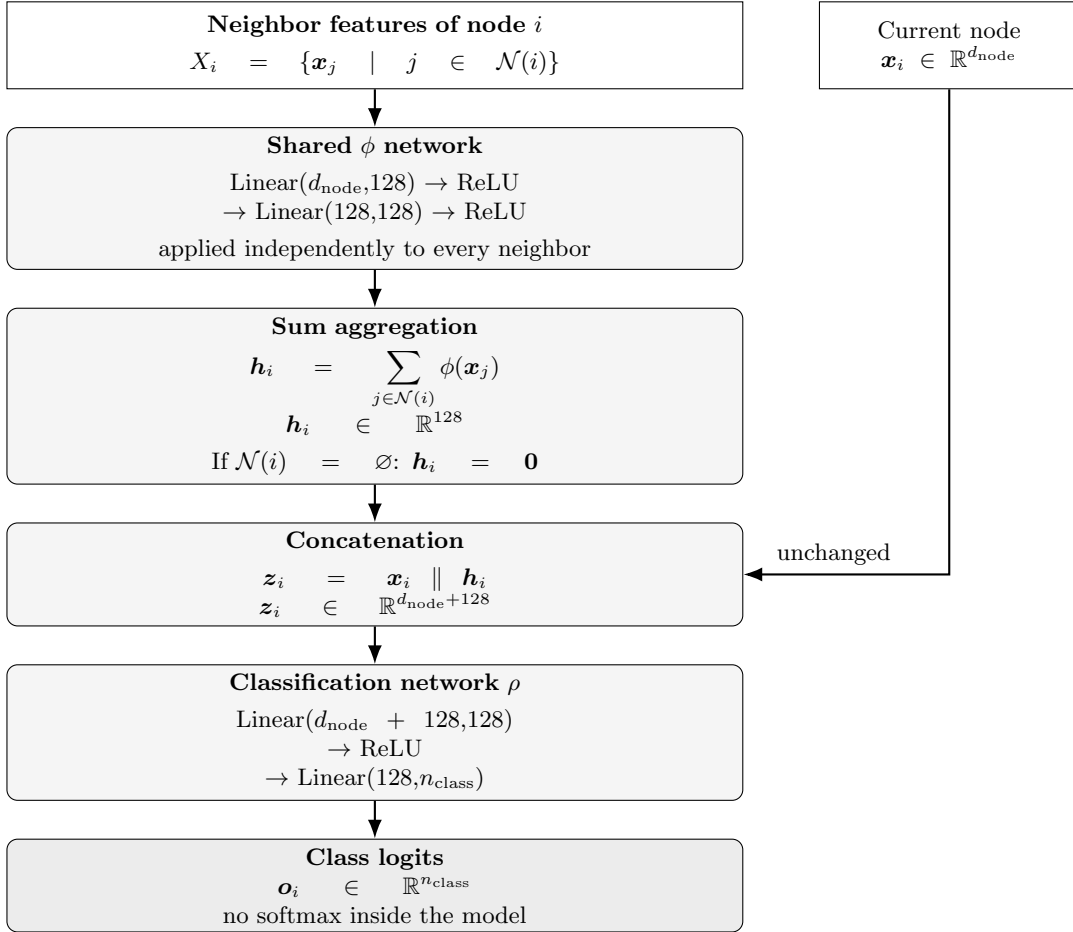


Figure 1: Architecture of the Deep Sets model used in our implementation.

In our implementation, the feature vector of the current node is concatenated with the result of the aggregation before ρ is applied. This was not done in the basic formulation of the original paper. In our case, however, we consider this choice reasonable because the features of the current node should also influence its neighborhood-dependent label. The network ρ , which is applied to the concatenated vector, is defined as

$$\text{Linear}(d_{\text{node}} + 128, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, n_{\text{class}}).$$

Both ϕ and ρ are implemented as simple feedforward networks with nonlinear activation functions. This choice was made to keep the implementation as simple as possible while remaining close to the fundamental Deep Sets structure. Alternatively, deeper networks, residual connections, normalization layers, or other activation functions such as GELU could have been used.

3.3.2 Set Transformer

Our Set Transformer implementation uses the same prepared neighborhood sets as the other models, but replaces the fixed sum aggregation of Deep Sets with attention-based encoding and pooling. For every node i , let $X_i = \{\mathbf{x}_j \mid j \in \mathcal{N}(i)\}$ denote its unordered neighborhood set, containing $m_i = |\mathcal{N}(i)|$ elements. The implemented node-level mapping is

$$\mathbf{o}_i = \rho(\mathbf{x}_i \parallel \text{PMA}(E(\phi(X_i)))) , \quad \mathbf{o}_i \in \mathbb{R}^{n_{\text{class}}} , \quad (3)$$

where E denotes either the SAB- or ISAB-based encoder. Thus, only the neighborhood X_i is processed by the Set Transformer. The features $\mathbf{x}_i$ of the current node bypass the attention-based encoder and are concatenated with the pooled neighborhood representation afterwards. This follows the same general node-classification structure as our Deep Sets implementation, while allowing the neighboring elements to interact before they are aggregated.

Before entering the encoder, every neighboring feature vector is projected into the internal feature space using

$$\phi = \text{Linear}(d_{\text{node}}, 128) \rightarrow \text{ReLU} .$$

The same projection is applied independently to every neighbor. It transforms the neighborhood from $X_i \in \mathbb{R}^{m_i \times d_{\text{node}}}$ to $X_i^{(0)} = \phi(X_i) \in \mathbb{R}^{m_i \times 128}$. The number of neighborhood elements therefore remains unchanged, while the feature dimension is projected to 128. No positional encodings are added because the neighborhood is processed as an unordered set. As shown in Fig. 2, the projected neighborhood follows either the SAB or ISAB encoder path before being aggregated by PMA. In contrast, the features of the current node follow a separate path directly to the final concatenation. Since neighborhood sizes can vary between nodes, the implementation processes every neighborhood individually using a temporary batch dimension of size one. This avoids the need for padding and attention masks.

All attention-based components use the Multihead Attention Block introduced in the theoretical section. In our implementation, each MAB has an internal feature dimension of 128 and uses four attention heads, resulting in 32 features per head. Its concrete structure is illustrated in Fig. 3. The row-wise feedforward network used within each block is defined as

$$\text{Linear}(128, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, 128) .$$

As shown in the figure, the attention and feedforward operations are each followed by a residual connection and layer normalization. Although all MABs use the same architecture, the individual block instances possess separate learnable parameters.

In Fig. 2 one can see that two alternative encoder variants were implemented. The SAB encoder consists of two consecutive Set Attention Blocks. Within each block, the neighborhood representations are used as queries, keys and values. Both blocks preserve the number of neighborhood elements and produce one 128-dimensional representation per neighbor. The two SABs are separate model components and therefore do not share their attention or feedforward parameters.

The alternative ISAB encoder also consists of two consecutive blocks. Each ISAB contains its own trainable inducing points with the dimension $m_{\text{ind}} \times 128$. These are learned model parameters rather than points selected from the input neighborhood and are initialized using Xavier-uniform initialization. As described in the theoretical section, the first MAB summarizes the neighborhood through the inducing points, while the second MAB transfers this information back to the original neighborhood elements. The output therefore again contains one 128-dimensional representation for every neighbor. Since each ISAB contains two MABs, the complete two-block encoder contains four separate MAB instances.

To investigate the influence of the number of inducing points, configurations with $m_{\text{ind}} \in \{8, 16, 32\}$ were evaluated and compared with the SAB variant. The number of encoder blocks, attention heads and internal features was held constant across all configurations. Regardless of the selected encoder, the output contains one context-dependent 128-dimensional representation for every neighbor and is subsequently processed by the same PMA layer. Our PMA implementation uses one trainable seed vector $S \in \mathbb{R}^{1 \times 128}$, which is initialized using Xavier-uniform initialization and shared across all node neighborhoods. The seed serves as the query of the MAB, while the encoded neighborhood $X_i^{(2)}$ provides its keys and values:

$$\mathbf{u}_i = \text{MAB}\left(S, X_i^{(2)}\right) \in \mathbb{R}^{1 \times 128}.$$

As illustrated in Fig. 2, the single seed vector transforms the variable-size encoder output into one fixed-size neighborhood representation. After removing the temporary batch and seed dimensions, $\mathbf{u}_i$ is represented as a 128-dimensional vector.

This implementation differs slightly from the general PMA formulation presented in the theoretical section, which applies an additional row-wise feedforward transformation to the encoded set before the MAB. Our implementation passes the encoder output directly to the MAB, although the row-wise feedforward network inside the MAB is still applied. Furthermore, no additional SAB is used after PMA. Since only one seed vector and therefore only one output representation is required, there are no multiple PMA output vectors whose interactions would have to be modelled.

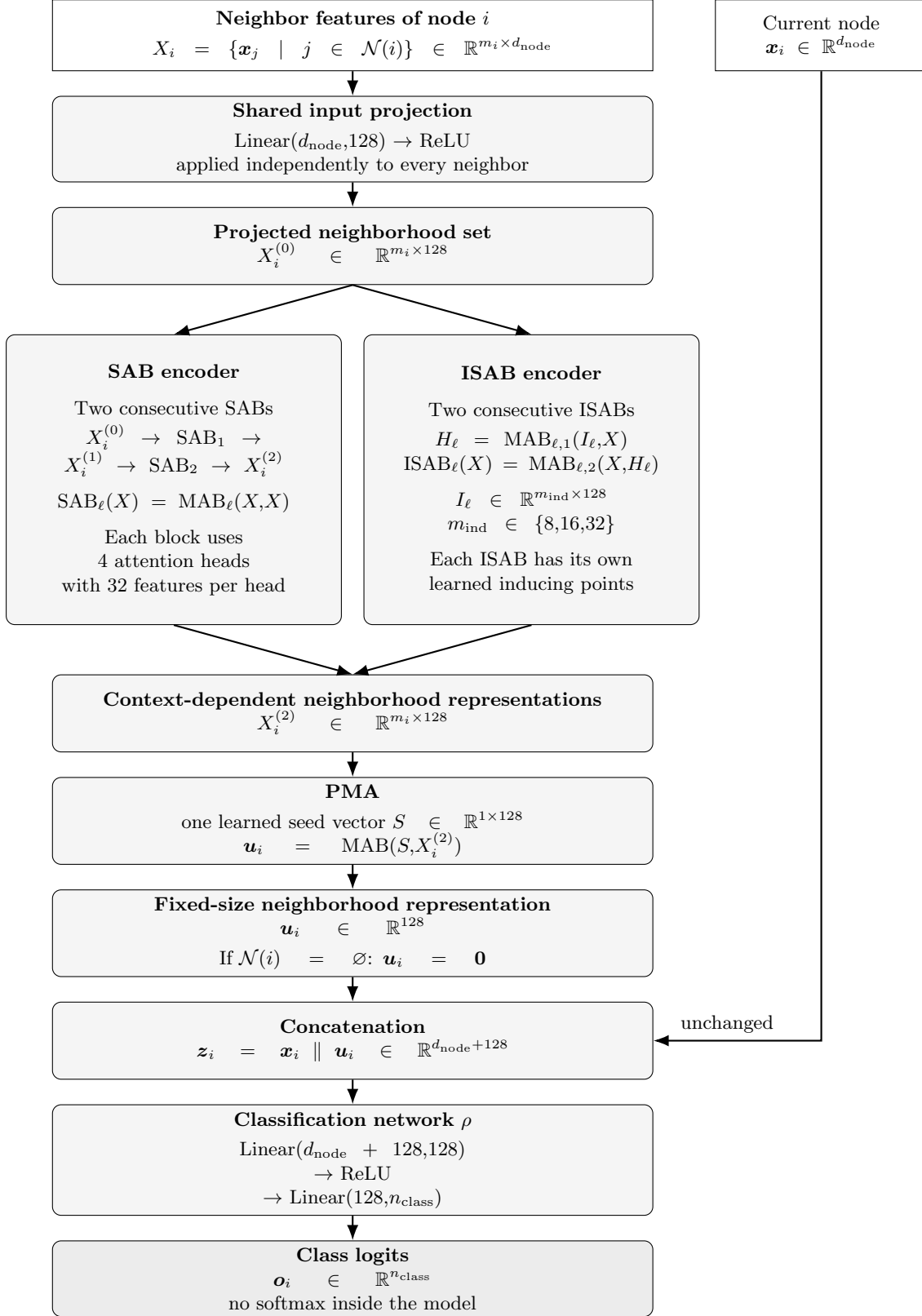


Figure 2: Architecture of the Set Transformer used for node classification.

As shown in the lower part of Fig. 2, the pooled neighborhood representation is concatenated with the unchanged

features of the current node. The resulting vector has the dimension $d_{\text{node}} + 128$ and is processed by the same classification-network structure as in our Deep Sets implementation:

$$\rho = \text{Linear}(d_{\text{node}} + 128, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, n_{\text{class}}).$$

The output contains one unnormalized class score for each possible class. As in the Deep Sets model, no softmax operation is included because the logits are passed directly to the cross-entropy loss during training.

For a node without neighbors, the input projection, encoder and PMA operations are skipped. Its neighborhood is instead represented by a zero vector $\mathbf{u}_i \in \mathbb{R}^{128}$, meaning that the prediction is based only on the features of the current node and the classification network.

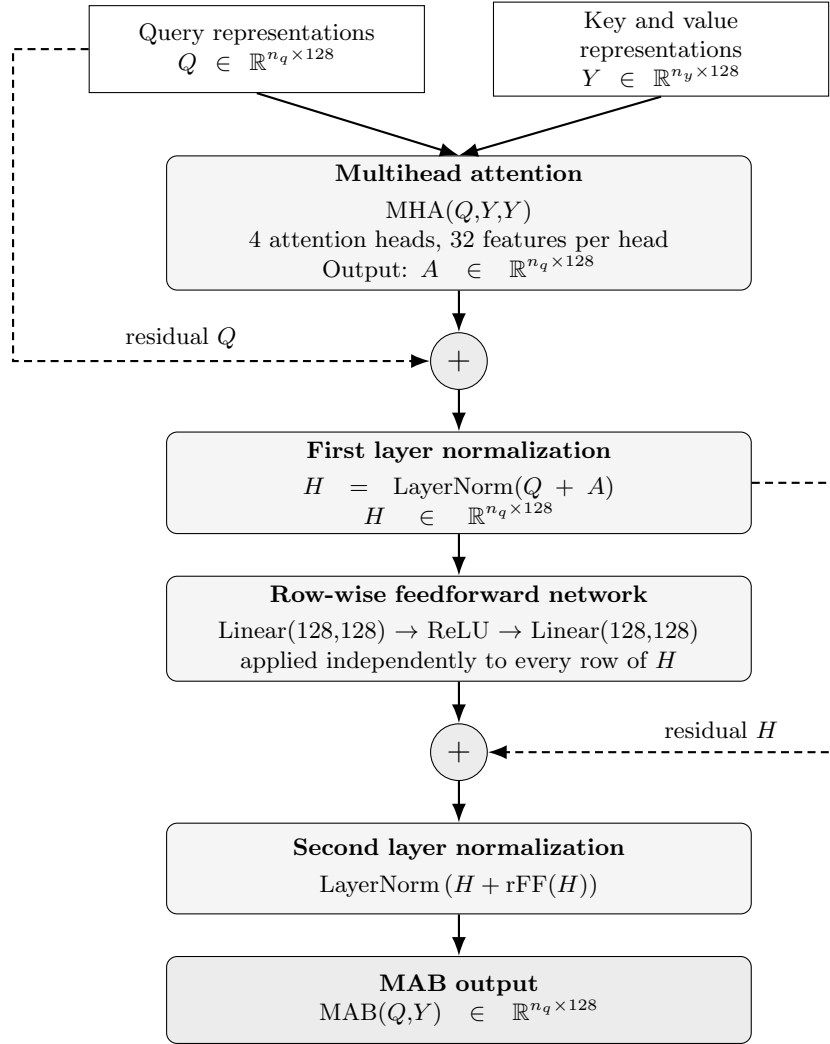


Figure 3: Internal structure of the MAB used by the SAB, ISAB and PMA components. The number of output rows is determined by the number of query vectors.

As discussed for the Deep Sets implementation, the internal feature dimension was fixed to 128 to increase comparability between the models. Apart from the encoder type and the number of inducing points described above,

the remaining architectural choices were held constant. Further alternatives, including other activation functions, additional encoder blocks, different numbers of attention heads, dropout, or multiple PMA seed vectors, were not examined in order to keep the experimental and computational scope manageable.

3.3.3 Janossy Pooling

Our implementation uses a sampling-based approximation of the Janossy Pooling described in the theory section. As with the previous models, $X_i = \{\mathbf{x}_j \mid j \in \mathcal{N}(i)\}$ denotes the unordered neighborhood of the node i currently being classified. Since the GRU used, unlike the set functions considered so far, depends on the order of its inputs, a random permutation π_r of the neighborhood is drawn for each forward pass. The calculation of the class logits for such a permutation can be expressed as

$$\mathbf{o}_i^{(r)} = \rho(\mathbf{x}_i \parallel g_{\text{GRU}}(\phi(\pi_r(X_i)))) , \quad \mathbf{o}_i^{(r)} \in \mathbb{R}^{n_{\text{class}}} . \quad (4)$$

Here, ϕ denotes the input projection and g_{GRU} denotes the mapping of an ordered neighborhood onto the final hidden state of the GRU. As with Deep Sets and the Set Transformer, only the neighboring features are processed by the actual neighborhood encoder. The features $\mathbf{x}_i$ of the current node initially remain unchanged and are only combined with the generated neighborhood representation before the classification network.

For each node and each call of **forward**, a new random order of its neighbors is generated independently. The feature vectors arranged in this way are then processed independently of one another using the same input projection:

$$\phi = \text{Linear}(d_{\text{node}}, 128) \rightarrow \text{ReLU} .$$

As a result, each neighbor is projected from the original feature dimension d_{node} into the 128-dimensional internal feature space. The number of neighbors and their randomly selected order are preserved in the process. For a neighborhood with m_i elements, this thus produces an ordered sequence of dimension $m_i \times 128$. The complete processing workflow is shown in Fig. 4. The figure illustrates both the path of a single random permutation and the different ways in which this path is used during training and evaluation.

The ordered sequence is processed by a single-layer, unidirectional GRU. Both the input dimension and the hidden dimension of the GRU are 128. No additional GRU layers, no bidirectional processing and no dropout are used. As **batch_first=True** is set and each neighborhood is processed individually, the sequence in the code is additionally given a batch dimension of size one. The GRU input therefore has the internal form $1 \times m_i \times 128$. Although the GRU generates an output for each sequence step, only the final hidden state of the single GRU layer is used for the neighborhood representation. This vector $\mathbf{h}_i^{(r)} \in \mathbb{R}^{128}$ summarises the neighborhood in the order determined by π_r . As the GRU updates its hidden states step by step, a different arrangement of the same neighbors can also lead to a different neighborhood representation. It is precisely this order dependency that is reduced by averaging across multiple permutations during evaluation. As shown in the lower part of Fig. 4, the final hidden state is concatenated with the unmodified features of the current node. The resulting vector has dimension $d_{\text{node}} + 128$ and

is processed by the classification network

$$\rho = \text{Linear}(d_{\text{node}} + 128, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, n_{\text{class}})$$

The output $\mathbf{o}_i^{(r)}$ contains an unnormalised class score for each possible class. A single call of **forward** therefore does not include a final softmax operation.

During training, exactly one random permutation is used for each node and each epoch. The logits generated from this are immediately evaluated using the cross-entropy loss function. This means that, during training, the model is presented with different sequences of the same neighborhoods without having to compute multiple complete GRU passes per node in every epoch. During validation and testing, however, the complete **forward** path is repeated R times. Each pass uses a newly generated permutation and produces its own logit vector $\mathbf{o}_i^{(r)}$. The logits are first transformed individually into class probabilities using Softmax and then averaged across the R permutations:

$$\bar{\mathbf{p}}_i = \frac{1}{R} \sum_{r=1}^R \text{softmax}(\mathbf{o}_i^{(r)}).$$

Consequently, in our implementation, it is not the hidden states or the logits that are averaged, but the resulting class probabilities. To calculate the NLL loss, the evaluation function returns the logarithm of these mean probabilities. Before taking the logarithm, the probabilities are clipped to at least 10^{-12} to avoid the numerically undefined expression $\log(0)$. The number of permutations used during the evaluation was varied using

$$R \in \{1, 5, 10, 20\}$$

A larger number can reduce the dependence of the prediction on individual random sequences and thus provide a better approximation of the Janossy mean. At the same time, the computational cost of validation and inference increases, as the full model path must be run again for each additional permutation. Even with a finite number of randomly drawn permutations, the result remains an approximation of the exact Janossy mean. If a node has no neighbors, the permutation, the input projection and the GRU are skipped. In this case, the neighborhood is represented by a zero vector in $\mathbb{R}^{128}$, so that the prediction depends only on the features of the current node and the classification network.

As with the other models, the internal feature dimension was set to 128 to enhance the comparability of the architectures. Apart from the number of permutations used during the evaluation, the architectural parameters were kept constant. Alternatively, the permutation-dependent function could, for example, have been implemented using a different sequence model. Similarly, additional GRU layers, a bidirectional GRU, different activation functions or a modified hidden dimension would have been possible. These variants were not investigated in order to keep the number of model configurations—and the associated implementation and computational effort—within the scope of this work.

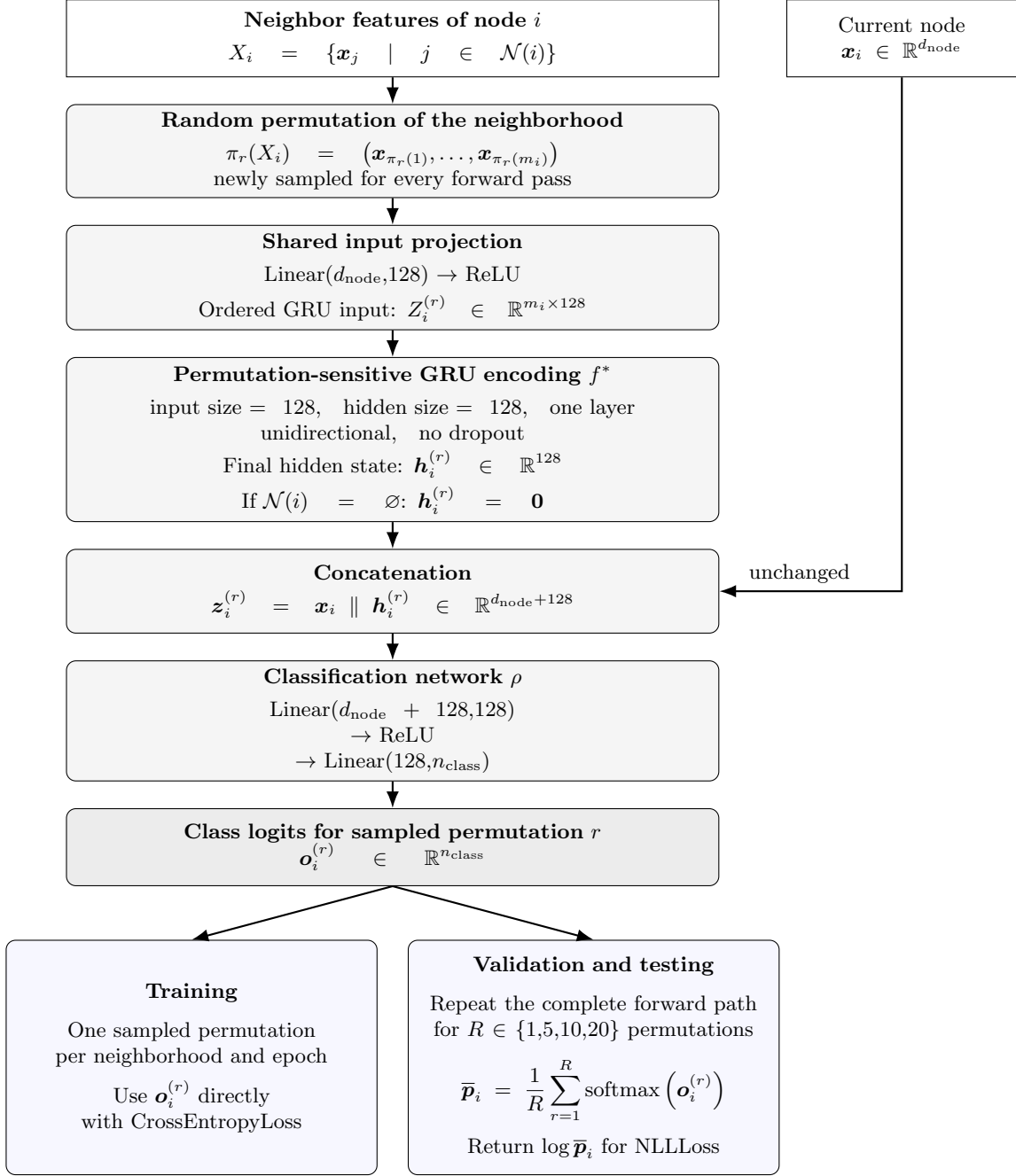


Figure 4: Architecture of the GRU-based approximate Janossy Pooling model.

3.3.4 PointNet++

Our PointNet++ implementation uses the same pre-computed neighbourhood sets as the other models and adapts the architecture—originally developed for point clouds—to the node classification task. For each node i to be classified, the features of its $m_i = |\mathcal{N}(i)|$ neighbours form the unordered input set:

$$X_i = \{\mathbf{x}_j \mid j \in \mathcal{N}(i)\} \in \mathbb{R}^{m_i \times d_{\text{node}}}.$$

This set is converted into a fixed neighbourhood vector by the PointNet++ encoder $g_{\text{PN}++}$. The features of the node currently being classified are retained separately and are only added back after its neighbourhood has been processed. The implemented mapping is therefore

$$\mathbf{o}_i = \rho(\mathbf{x}_i \parallel g_{\text{PN}++}(X_i)), \quad \mathbf{o}_i \in \mathbb{R}^{n_{\text{class}}}, \quad (5)$$

where $\mathbf{o}_i$ contains the unnormalised class scores for node i . The complete flow of the architecture is shown in Fig. 5.

One important adaptation compared with the original scope of application of PointNet++ concerns the interpretation of the input features. The datasets used do not have separate spatial coordinates and point attributes. Therefore, three different representations are first derived from the neighbouring features. The unmodified features are used as metric coordinates:

$$M_i^{(0)} = X_i \in \mathbb{R}^{m_i \times d_{\text{node}}}.$$

FPS and kNN therefore calculate their Euclidean distances in the original feature space. In addition, compact local coordinates are generated:

$$L_i^{(0)} = \text{Linear}(d_{\text{node}}, 16, \text{bias} = \text{False})(X_i) \in \mathbb{R}^{m_i \times 16}.$$

These learnt coordinates are not used to select the centroids or their neighbours, but are used later to calculate the relative coordinates within the local clusters. In addition, the actual point features are defined by

$$F_i^{(0)} = \text{ReLU}(\text{Linear}(d_{\text{node}}, 128)(X_i)) \in \mathbb{R}^{m_i \times 128}.$$

The three representations and their respective roles are summarized at the top of Fig. 5. In particular, the original features determine the metric used for FPS and kNN, whereas the learned local coordinates are used only to represent relative positions within the resulting groups.

As illustrated by the first dashed block in Fig. 5, the first set-abstraction level consists of deterministic FPS, kNN grouping and a local PointNet. FPS is applied to the metric coordinates $M_i^{(0)}$. While the first centroid can generally be selected arbitrarily, our implementation starts with the point furthest from the mean of the set. Each subsequent centroid maximises its minimum squared Euclidean distance to the previously selected centroids. This provides a deterministic starting point while retaining the coverage property of FPS. The requested number of centroids and the effective group size are limited by the number of available points:

$$n_1 = \min(c_1, m_i), \quad K_1 = \min(8, m_i).$$

The kNN search uses the same metric coordinates as FPS. For each resulting group, the 16-dimensional local coordinates are expressed relative to the selected centroid and concatenated with the corresponding 128-dimensional point features. This produces the grouped tensor

$$Z_i^{(1)} \in \mathbb{R}^{n_1 \times K_1 \times 144}.$$

The grouped points are processed by the shared MLP

$$\text{Linear}(144,64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64,64) \rightarrow \text{ReLU} \rightarrow \text{Linear}(64,128) \rightarrow \text{ReLU}.$$

Its parameters are shared across all points and groups. Element-wise max pooling over each group produces the new centroid features

$$F_i^{(1)} \in \mathbb{R}^{n_1 \times 128}.$$

The metric and local coordinates of the selected centroids are retained as $M_i^{(1)}$ and $L_i^{(1)}$ for the following abstraction level.

The second dashed block in Fig. 5 applies the same sequence to the centroids produced by the first level. FPS is now applied to $M_i^{(1)}$ and the effective numbers of centroids and grouped elements are

$$n_2 = \min(c_2, n_1), \quad K_2 = \min(4, n_1).$$

As in the first level, the relative 16-dimensional local coordinates are concatenated with the 128-dimensional point features. The resulting 144-dimensional vectors are processed by

$$\text{Linear}(144,128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128,128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128,128) \rightarrow \text{ReLU}.$$

Max pooling within each group produces the second-level centroid features

$$F_i^{(2)} \in \mathbb{R}^{n_2 \times 128}.$$

FPS and kNN are discrete index-selection operations and are therefore executed without an autograd graph. The projected coordinates and point features selected through these indices, as well as the shared MLPs applied to them, remain part of the trainable model.

The third dashed block in Fig. 5 represents the global abstraction. Unlike the preceding levels, it does not perform any further sampling or grouping. Instead, the mean of the remaining local coordinates is used as a global reference point:

$$\bar{\ell}_i = \frac{1}{n_2} \sum_{r=1}^{n_2} L_{i,r}^{(2)}.$$

The centred local coordinates are concatenated with the corresponding point features and processed by

$$\text{Linear}(144,128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128,128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128,128) \rightarrow \text{ReLU}.$$

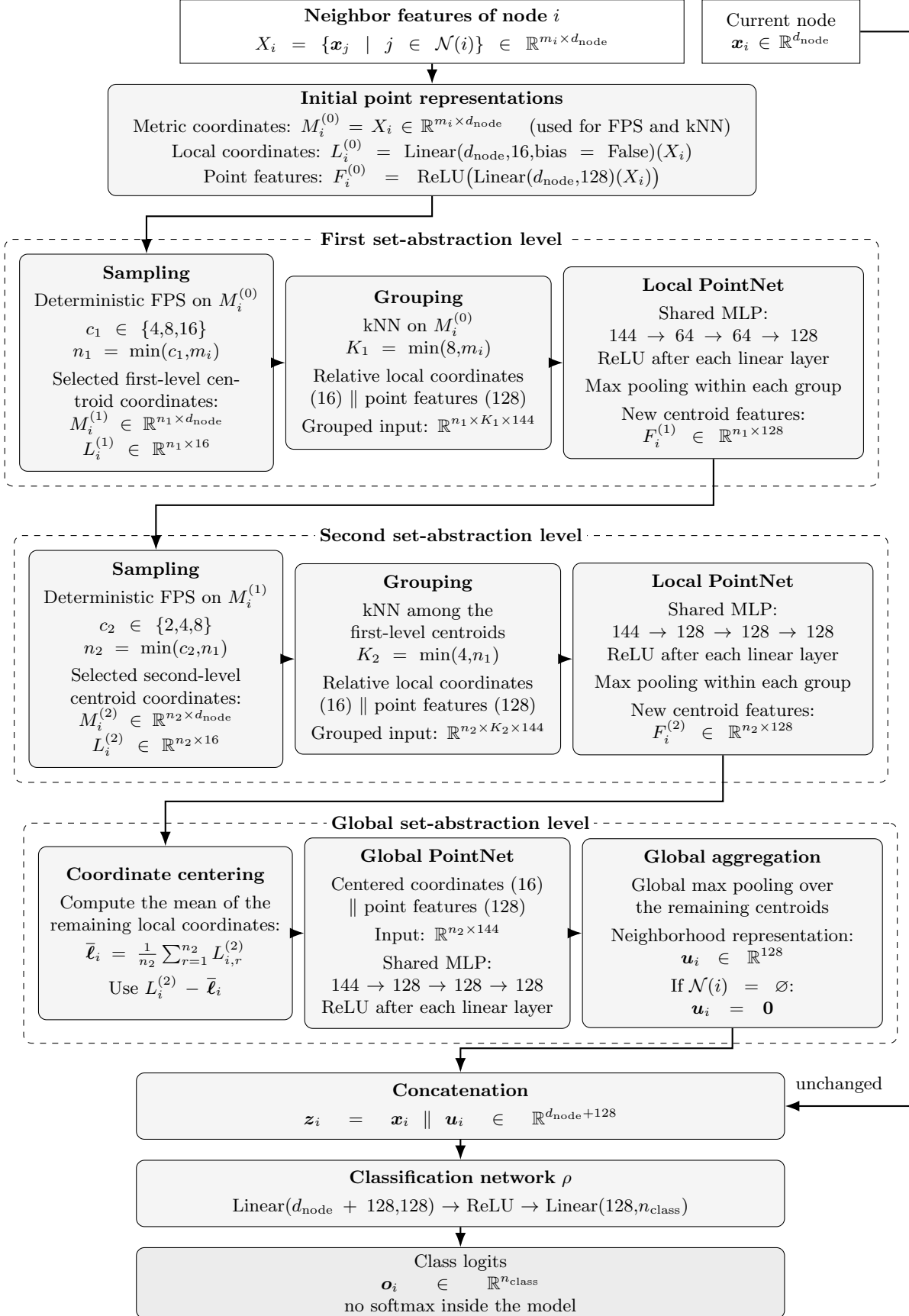


Figure 5: Architecture of the PointNet++ model used for node classification.

Global max pooling over the resulting representations produces the fixed-size neighbourhood vector

$$\mathbf{u}_i = g_{\text{PN}++}(X_i) \in \mathbb{R}^{128}.$$

The global reference point is therefore not a point selected through FPS, but the mean of the learned local coordinates. The neighbourhood vector is then combined with the unchanged features of the node currently being classified:

$$\mathbf{z}_i = \mathbf{x}_i \parallel \mathbf{u}_i \in \mathbb{R}^{d_{\text{node}}+128}.$$

As with the other models, the final classification network is defined as

$$\rho = \text{Linear}(d_{\text{node}} + 128, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, n_{\text{class}}).$$

The output contains a logit for each possible class. No additional softmax function is applied within the model, as the logits are passed directly to the cross-entropy loss function during training. If a node has no neighbours, the PointNet++ abstraction layers are skipped and the neighbourhood is determined by

$$\mathbf{u}_i = \mathbf{0} \in \mathbb{R}^{128}.$$

In this case, the prediction is based solely on the features of the current node and the classification network.

As with the other model implementations, the internal feature dimension was set to 128 to improve comparability. The dimension of the local coordinates was set to 16, the number of neighbours in the first set abstraction level to 8 and in the second level to 4. Only the numbers of centroid points were varied. The experiments investigated the following configurations:

$$(c_1, c_2) \in \{(4, 2), (8, 4), (16, 8)\}.$$

Our implementation therefore uses hierarchical single-scale grouping based on kNN and does not include any additional MSG or MRG branches. A radius-based ball query would have required appropriate, dataset-dependent radii for both the synthetic data and CiteSeer, as their feature spaces and scales differ a lot. Therefore, kNN was used to maintain a largely uniform group size. Furthermore, following the max pooling of the local PointNets, no additional γ network is applied. Other possible variations—such as different MLP structures, activation functions, coordinate dimensions, neighbourhood sizes or additional levels of abstraction—were not investigated in order to limit the number of model configurations and the computational cost within the scope of this work.

3.4 Experiment execution

All comparison runs were executed through `run_all_experiments.sh`. The script applies the same general training configuration to both the synthetic and CiteSeer datasets. Each model uses an internal feature dimension of 128, a learning rate of 0.001 and a maximum of 50 training epochs. Every model configuration is trained independently

using the five seeds

$$\{42, 43, 44, 45, 46\}.$$

For each dataset, the evaluated configurations comprise one Deep Sets model, one SAB and three ISAB variants of the Set Transformer, four Janossy GRU variants and three PointNet++ centroid configurations. This results in

$$2 \text{ datasets} \times 12 \text{ model configurations} \times 5 \text{ seeds} = 120$$

individual training runs.

Each run is stored in a separate directory containing the trained checkpoint, the training log, the executed command and the extracted evaluation metrics. The script additionally combines the results in `summary.csv` and records the execution status of each run in `run_manifest.csv`. Runs that have already been completed successfully are skipped unless their repeated execution is explicitly requested. The shell script only coordinates the individual experiments and result collection; the model training, validation-based checkpoint selection and test evaluation remain implemented in the corresponding Python files.

4 Experimental results

4.1 Overall and class-specific classification performance

Table 1 shows the classification results for the model configurations under investigation, averaged across five seeds. Across both datasets, the mean test accuracies range from 0.718 to 0.778. The Weighted F1 scores range from 0.699 to 0.778, whilst the Macro F1 covers a slightly wider range, with values between 0.642 and 0.778. Overall, the differences between the architectures are minor on the synthetic dataset. On CiteSeer, more pronounced differences between the model families emerge, particularly for the Macro-F1.

On the synthetic dataset, the test accuracies of all configurations fall within the relatively narrow range of 0.754 to 0.778. Janossy GRU achieves the highest mean test accuracy with $P = 20$ and 0.778 ± 0.025 . This is followed by PointNet++ with $(c_1, c_2) = (16, 8)$ and 0.775 ± 0.019 and Janossy GRU with $P = 10$ and 0.774 ± 0.025 . Deep Sets also achieves a comparable result with 0.770 ± 0.025 . The differences between these leading configurations are therefore less than one percentage point and are small in relation to the respective standard deviations. It is therefore not possible to deduce any clear superiority of one configuration over another from the mean values alone.

Within the Janossy-GRU configurations, the average test accuracy on the synthetic dataset increases gradually with the number of inference permutations, from 0.766 for $P = 1$ to 0.778 for $P = 20$. A similar, albeit equally small, increase is observed for PointNet++, where the mean test accuracy rises from 0.765 for $(c_1, c_2) = (4, 2)$ to 0.775 for $(c_1, c_2) = (16, 8)$. The latter PointNet++ configuration also exhibits comparatively low variance, with a standard deviation of 0.019. The Set Transformer configurations do not achieve better results on the synthetic dataset than Deep Sets or the best Janossy and PointNet++ configurations. SAB and ISAB with 16 and 32 in-

ducing points achieve similar test accuracies between 0.766 and 0.767. ISAB with 8 inducing points achieves the lowest mean test accuracy on this dataset, at 0.754 ± 0.023 . Increasing the number of inducing points from 16 to 32, however, does not lead to any further improvement.

Table 1: Mean classification results across five seeds. Values are reported as mean $\pm$ sample standard deviation. P denotes the number of inference permutations used by Janossy Pooling, (c_1, c_2) the numbers of centroids in the two PointNet++ abstraction levels and m_{ind} the number of inducing points used by ISAB.

Dataset	Model	Configuration	Test accuracy	Macro-F1	Weighted F1	Best validation accuracy
CiteSeer	Deep Sets	–	0.759 ± 0.020	0.715 ± 0.019	0.749 ± 0.016	0.776 ± 0.034
	Set Transformer	SAB	0.723 ± 0.019	0.642 ± 0.018	0.699 ± 0.019	0.750 ± 0.028
		ISAB, $m_{\text{ind}} = 8$	0.723 ± 0.018	0.661 ± 0.036	0.707 ± 0.022	0.742 ± 0.026
		ISAB, $m_{\text{ind}} = 16$	0.718 ± 0.016	0.667 ± 0.039	0.707 ± 0.024	0.738 ± 0.026
		ISAB, $m_{\text{ind}} = 32$	0.718 ± 0.016	0.667 ± 0.036	0.707 ± 0.022	0.748 ± 0.019
	Janossy GRU	$P = 1$	0.757 ± 0.027	0.718 ± 0.038	0.750 ± 0.028	0.780 ± 0.027
		$P = 5$	0.757 ± 0.023	0.714 ± 0.035	0.749 ± 0.026	0.781 ± 0.027
		$P = 10$	0.763 ± 0.020	0.721 ± 0.032	0.755 ± 0.022	0.778 ± 0.028
		$P = 20$	0.762 ± 0.013	0.723 ± 0.021	0.755 ± 0.014	0.780 ± 0.029
	PointNet++	$(c_1, c_2) = (4, 2)$	0.722 ± 0.013	0.665 ± 0.031	0.707 ± 0.018	0.748 ± 0.009
		$(c_1, c_2) = (8, 4)$	0.720 ± 0.014	0.651 ± 0.024	0.700 ± 0.017	0.747 ± 0.008
		$(c_1, c_2) = (16, 8)$	0.720 ± 0.014	0.656 ± 0.025	0.702 ± 0.018	0.748 ± 0.010
Synthetic	Deep Sets	–	0.770 ± 0.025	0.770 ± 0.025	0.769 ± 0.025	0.774 ± 0.019
	Set Transformer	SAB	0.767 ± 0.038	0.766 ± 0.039	0.766 ± 0.039	0.779 ± 0.022
		ISAB, $m_{\text{ind}} = 8$	0.754 ± 0.023	0.753 ± 0.023	0.753 ± 0.023	0.778 ± 0.011
		ISAB, $m_{\text{ind}} = 16$	0.766 ± 0.032	0.765 ± 0.033	0.765 ± 0.033	0.777 ± 0.016
		ISAB, $m_{\text{ind}} = 32$	0.766 ± 0.031	0.766 ± 0.031	0.766 ± 0.031	0.777 ± 0.019
	Janossy GRU	$P = 1$	0.766 ± 0.019	0.766 ± 0.019	0.766 ± 0.019	0.771 ± 0.011
		$P = 5$	0.772 ± 0.029	0.772 ± 0.029	0.772 ± 0.029	0.770 ± 0.012
		$P = 10$	0.774 ± 0.025	0.774 ± 0.025	0.774 ± 0.025	0.769 ± 0.015
		$P = 20$	0.778 ± 0.025	0.778 ± 0.026	0.778 ± 0.026	0.768 ± 0.014
	PointNet++	$(c_1, c_2) = (4, 2)$	0.765 ± 0.024	0.763 ± 0.027	0.763 ± 0.027	0.778 ± 0.017
		$(c_1, c_2) = (8, 4)$	0.768 ± 0.026	0.765 ± 0.029	0.765 ± 0.029	0.779 ± 0.016
		$(c_1, c_2) = (16, 8)$	0.775 ± 0.019	0.774 ± 0.019	0.774 ± 0.019	0.779 ± 0.019

Test accuracy, macro-F1 and weighted-F1 differ only slightly across the individual configurations on the synthetic dataset. The overall accuracy and the F1 scores aggregated across both classes therefore lead to a similar assessment of the model configurations. This general observation is largely supported by the class-specific results in Table 2. Even when the two classes are considered separately, the selected configurations achieve comparable overall precision and recall values. At the same time, a consistent pattern emerges across all four models, whereby precision is higher for class 1, whilst recall is higher for class 0. This relationship is particularly pronounced in PointNet++. Here, the recall for class 0 is 0.836 ± 0.088 , whereas for class 1 it is only 0.718 ± 0.080 . The comparatively high standard deviations of these recall values also indicate greater variation between seeds. In contrast, Janossy GRU and the Set Transformer achieve the most balanced precision and recall values across the two classes.

Table 2: Class-specific test precision and recall for the synthetic dataset. For each model family, the configuration with the highest mean Macro-F1 was selected. Values are reported as mean $\pm$ sample standard deviation across five seeds.

Model	Configuration	Class	Precision	Recall
Deep Sets	–	0	0.745 ± 0.038	0.810 ± 0.019
		1	0.800 ± 0.015	0.732 ± 0.056
Set Transformer	SAB	0	0.753 ± 0.042	0.784 ± 0.064
		1	0.786 ± 0.048	0.751 ± 0.060
Janossy GRU	$P = 20$	0	0.761 ± 0.032	0.799 ± 0.026
		1	0.797 ± 0.022	0.758 ± 0.043
PointNet++	$(c_1, c_2) = (16, 8)$	0	0.743 ± 0.038	0.836 ± 0.088
		1	0.829 ± 0.062	0.718 ± 0.080

On CiteSeer, the mean test accuracies range between 0.718 and 0.763 (Table 1). Janossy GRU achieves the highest result with $P = 10$ and 0.763 ± 0.020 , closely followed by $P = 20$ with 0.762 ± 0.013 and Deep Sets with 0.759 ± 0.020 . Janossy GRU with $P = 20$ exhibits comparatively low variance whilst also achieving the highest macro-F1 of 0.723 ± 0.021 . However, the differences between Deep Sets and the Janossy configurations remain small on CiteSeer relative to the observed variance.

PointNet++ achieves test accuracies on CiteSeer of between 0.720 and 0.722, which is approximately four percentage points lower than the best Janossy configurations. Its Macro-F1 values range between 0.651 and 0.665, whilst the Weighted-F1 values range between 0.700 and 0.707. A larger number of centroids does not lead to any consistent improvement on CiteSeer. The smallest configuration $(c_1, c_2) = (4, 2)$ achieves the highest Macro-F1 and Weighted-F1 within this model family. The Set Transformer configurations achieve test accuracies on CiteSeer ranging from 0.718 to 0.723. Whilst the accuracy of the SAB and ISAB variants differs only slightly, the Macro-F1 of the SAB configuration is particularly low at 0.642 ± 0.018 . The ISAB configurations with 16 and 32 inducing points each achieve a Macro-F1 of approximately 0.667, without simultaneously achieving a higher test accuracy. The number of inducing points therefore has no monotonic effect on the classification metrics under consideration.

In contrast to the synthetic dataset, the Weighted F1 on CiteSeer is higher than the Macro F1 across all configurations. The difference is particularly pronounced for PointNet++ and the Set Transformer. The two F1 metrics thus demonstrate that the evaluation of the models on CiteSeer depends more heavily on whether all classes are treated equally or weighted according to their frequency. The class-specific results in Table 3 show relatively big performance differences across the six classes. Class 0 is particularly notable. Its recall is substantially lower than that of the other classes for all four model families. Janossy GRU achieves the highest recall for this class, at 0.323 ± 0.114 . This is followed by Deep Sets at 0.285 ± 0.153 , the Set Transformer at 0.210 ± 0.142 and PointNet++ at 0.172 ± 0.111 . The sometimes high standard deviations of the precision and recall values also show that the detection of class 0 varies comparatively widely between the seeds. For classes 1 to 5, on the other hand, the recall values are predominantly between 0.702 and 0.842. Janossy GRU achieves particularly balanced and predominantly high class-specific results overall. PointNet++ and the Set Transformer exhibit not only low recall for class 0 but

also comparatively low precision values for class 1. The poor results for individual classes are reflected in the lower macro-F1 score. The class-specific analysis also shows that the aggregated metrics sometimes mask clear differences in performance between the classes.

Table 3: Class-specific test precision and recall for CiteSeer. For each model family, the configuration with the highest mean Macro-F1 was selected. Values are reported as mean $\pm$ sample standard deviation across five seeds.

Model	Configuration	Class	Precision	Recall
Deep Sets	–	0	0.696 ± 0.232	0.285 ± 0.153
		1	0.675 ± 0.030	0.756 ± 0.023
		2	0.753 ± 0.055	0.761 ± 0.056
		3	0.774 ± 0.035	0.829 ± 0.056
		4	0.813 ± 0.024	0.842 ± 0.051
		5	0.821 ± 0.047	0.812 ± 0.026
Set Transformer	ISAB, $m_{\text{ind}} = 32$	0	0.485 ± 0.323	0.210 ± 0.142
		1	0.607 ± 0.038	0.702 ± 0.054
		2	0.729 ± 0.042	0.740 ± 0.048
		3	0.785 ± 0.037	0.777 ± 0.030
		4	0.744 ± 0.066	0.818 ± 0.050
		5	0.778 ± 0.061	0.776 ± 0.079
Janossy GRU	$P = 20$	0	0.599 ± 0.097	0.323 ± 0.114
		1	0.671 ± 0.037	0.753 ± 0.035
		2	0.744 ± 0.022	0.782 ± 0.037
		3	0.794 ± 0.023	0.803 ± 0.026
		4	0.823 ± 0.027	0.842 ± 0.061
		5	0.840 ± 0.038	0.824 ± 0.060
PointNet++	$(c_1, c_2) = (4, 2)$	0	0.727 ± 0.243	0.172 ± 0.111
		1	0.632 ± 0.041	0.725 ± 0.059
		2	0.754 ± 0.026	0.740 ± 0.080
		3	0.722 ± 0.038	0.806 ± 0.047
		4	0.750 ± 0.029	0.822 ± 0.049
		5	0.788 ± 0.090	0.749 ± 0.084

The best validation accuracies show a similar overall pattern (Table 1). On the synthetic dataset, they range from 0.768 to 0.779. On CiteSeer, Deep Sets and Janossy GRU achieve values between 0.776 and 0.781, whereas PointNet++ and the Set Transformer range between 0.738 and 0.750. These values represent the maximum validation accuracy across the training epochs and were used to select the model checkpoint evaluated on the test set.

4.2 CiteSeer classification performance by node degree

The analysis of classification accuracy by node degree is restricted to CiteSeer. In the synthetic dataset, each point has exactly $k = 10$ neighbours due to the kNN construction used. As there are therefore no distinct degree groups, a degree-dependent evaluation would not be meaningful. CiteSeer, on the other hand, exhibits varying node degrees and therefore allows a comparison of model performance as a function of neighbourhood size. As shown in Table 4, the degree groups in the CiteSeer test set are very unevenly distributed. Of an average of 333 test nodes, 214.6 – or approximately 64.4% – belong to the group with a node degree of 1–2. A further 80.2 nodes, or approximately

24.1%, belong to the 3–5 group. In contrast, the groups with degrees 0 and > 10 each contain, on average, only about six test nodes. The results for these small groups therefore depend more heavily on individual nodes and the composition of the respective test split. Accordingly, the standard deviation must be taken into account in particular when interpreting them. The overall accuracy on the test set, by contrast, is mainly determined by the well-represented groups 1–2 and 3–5.

Table 4: Numbers of CiteSeer test nodes in the different degree groups. Values are reported as mean $\pm$ sample standard deviation across the five seed-dependent test splits.

Node degree	Number of test nodes
0	5.8 ± 2.0
1–2	214.6 ± 9.5
3–5	80.2 ± 5.1
6–10	26.2 ± 5.4
> 10	6.2 ± 3.1

Table 5 shows that the test accuracy generally increases with node degree. For isolated nodes, the average accuracy across all configurations ranges from just 0.445 to 0.480. At the same time, the results for this group, with standard deviations between 0.140 and 0.220, show a comparatively high degree of variation. In the degree group 1–2, the accuracy rises to values between 0.686 and 0.751. For the groups 3–5 and 6–10, ranges of 0.751 to 0.796 and 0.781 to 0.820 respectively are achieved. For nodes with a degree greater than 10, the previous increase in accuracy does not continue consistently. The values here range between 0.756 and 0.818, whilst the standard deviations in some cases reach up to 0.131. Overall, the differences between the degree groups are greater than most of the differences between the model configurations within a single degree group. However, no reliable conclusions can be drawn from the edge groups. Therefore, the following assessments are limited to the other groups.

Table 5: CiteSeer test accuracy grouped by node degree. Values are reported as mean $\pm$ sample standard deviation across five seeds. P denotes the number of inference permutations used by Janossy Pooling, (c_1, c_2) the numbers of centroids in the two PointNet++ abstraction levels and m_{ind} the number of inducing points used by ISAB.

Model	Configuration	Test accuracy by node degree				
		0	1–2	3–5	6–10	> 10
Deep Sets	–	0.455 ± 0.140	0.742 ± 0.024	0.796 ± 0.036	0.819 ± 0.070	0.818 ± 0.125
Set Transformer	SAB	0.480 ± 0.159	0.698 ± 0.027	0.776 ± 0.030	0.781 ± 0.047	0.800 ± 0.131
	ISAB, $m_{\text{ind}} = 8$	0.480 ± 0.159	0.692 ± 0.033	0.781 ± 0.028	0.817 ± 0.039	0.800 ± 0.131
	ISAB, $m_{\text{ind}} = 16$	0.480 ± 0.159	0.686 ± 0.012	0.782 ± 0.041	0.808 ± 0.050	0.800 ± 0.131
	ISAB, $m_{\text{ind}} = 32$	0.455 ± 0.140	0.686 ± 0.014	0.783 ± 0.028	0.813 ± 0.072	0.800 ± 0.131
Janossy GRU	$P = 1$	0.470 ± 0.220	0.748 ± 0.026	0.784 ± 0.031	0.806 ± 0.076	0.785 ± 0.077
	$P = 5$	0.470 ± 0.220	0.745 ± 0.025	0.789 ± 0.024	0.795 ± 0.040	0.803 ± 0.096
	$P = 10$	0.470 ± 0.220	0.751 ± 0.018	0.794 ± 0.037	0.819 ± 0.086	0.785 ± 0.077
	$P = 20$	0.470 ± 0.220	0.750 ± 0.018	0.789 ± 0.018	0.817 ± 0.047	0.803 ± 0.096
PointNet++	$(c_1, c_2) = (4, 2)$	0.445 ± 0.184	0.698 ± 0.024	0.781 ± 0.035	0.793 ± 0.059	0.756 ± 0.070
	$(c_1, c_2) = (8, 4)$	0.470 ± 0.158	0.700 ± 0.027	0.751 ± 0.049	0.820 ± 0.079	0.772 ± 0.131
	$(c_1, c_2) = (16, 8)$	0.445 ± 0.184	0.695 ± 0.025	0.768 ± 0.037	0.808 ± 0.071	0.818 ± 0.125

The most pronounced differences between models occur in the heavily populated degree group 1–2. Here, Janossy GRU achieves accuracy values between 0.745 and 0.751, depending on the number of inference permutations. Deep Sets achieves a similar result of 0.742. PointNet++ falls below this, with values between 0.695 and 0.700, whilst the Set Transformer achieves values between 0.686 and 0.698. This order largely corresponds to the overall results on CiteSeer considered previously. As around 64.4% of the test nodes belong to this degree group, it accounts for a particularly large proportion of the overall accuracy. At higher node degrees, the differences between the model families are smaller. In the 6–10 group, all configurations fall between 0.781 and 0.820. PointNet++ with $(c_1, c_2) = (8, 4)$ achieves the highest mean here at 0.820 ± 0.079 . Deep Sets and Janossy GRU with $P = 10$ follow, each with 0.819. The Set Transformer achieves 0.817 ± 0.039 with ISAB and 8 inducing points. However, these differences are small and lie within the respective standard deviations.

Within the individual model families, there is no monotonic relationship between the scope of the configuration and the accuracy across the various degree groups. In the case of Janossy GRU, a higher number of inference permutations does not lead to an improvement in every group. Similarly, increasing the size of the two centroid sets in PointNet++ does not consistently yield higher values. For example, in the 6–10 group, the medium PointNet++ configuration achieves a higher value of 0.820 than the largest configuration, which has a value of 0.808. With the Set Transformer, an increasing number of inducing points also does not lead to a consistent improvement. The node-degree-dependent results thus confirm the trend already observed in the general classification evaluation, namely that more complex configurations do not consistently achieve higher accuracies.

4.3 Computational efficiency

In addition to classification performance, computational efficiency is also relevant for the practical applicability of the models. Table 6 therefore summarises the training times, inference times per node and the number of trainable parameters for all configurations examined. The time measurements are given as the mean and the sample standard deviation across five seeds.

In Table 6, the number of trainable parameters is initially used to classify model size. Deep Sets has the lowest number of parameters on both datasets. This is followed by Janossy GRU, PointNet++, the SAB variant and, finally, the ISAB variants. On CiteSeer, the model size ranges from 981 894 parameters for Deep Sets to 1 471 622 parameters for ISAB with 32 inducing points. On the synthetic dataset, the corresponding range is between 35 458 and 525 186 parameters. The clearly higher values on CiteSeer result in particular from the greater dimensionality of the input features. The absolute number of parameters should therefore primarily be compared within the same dataset. Within Janossy GRU and PointNet++, the number of parameters remains constant across all configurations examined, whilst in ISAB it increases slightly with the number of trainable inducing points. These differences are directly determined by the respective architectures and are therefore not a performance result on their own. However, they form an important basis for classifying training and inference times.

Deep Sets has the shortest training time on both datasets. The model takes 132.6 ± 1.0 seconds on CiteSeer

and 58.7 ± 0.4 seconds on the synthetic dataset. By contrast, the longest training times are recorded for the ISAB variants. On CiteSeer, these range from 969.2 ± 9.9 to 1014.6 ± 37.8 seconds. On the synthetic dataset, they are approximately 903 to 910 seconds. The SAB variant is faster than ISAB, taking 567.3 ± 4.0 seconds on CiteSeer and 517.9 ± 2.5 seconds on the synthetic dataset. However, it still takes considerably longer than Deep Sets. The training times of PointNet++ fall between those of Deep Sets and the Set Transformer variants. On CiteSeer, they increase from 454.9 ± 5.7 to 489.9 ± 16.1 seconds as the number of centroids rises. On the synthetic dataset, an increase from 345.7 ± 0.8 to 424.6 ± 2.4 seconds can be observed. For Janossy GRU, too, the training time depends a lot on the configuration under investigation. As validation is included in the time measurement, a larger number of inference permutations increases the measured time. On CiteSeer, the training time increases from 222.3 ± 6.5 seconds for $P = 1$ to 517.2 ± 11.5 seconds for $P = 20$. On the synthetic dataset, it increases correspondingly from 236.7 ± 1.7 to 485.9 ± 0.7 seconds. Janossy GRU is therefore among the faster models, particularly for small and medium values of P . For $P = 20$, its training time is already close to that of the SAB variant and exceeds that of all PointNet++ configurations.

Table 6: Efficiency results across five seeds. Training and inference times are reported as mean $\pm$ sample standard deviation. Training time includes all 50 training epochs and the validation performed after each epoch. The number of trainable parameters is constant across seeds. P denotes the number of inference permutations used by Janossy Pooling, (c_1, c_2) the numbers of centroids in the two PointNet++ abstraction levels and m_{ind} the number of inducing points used by ISAB.

Dataset	Model	Configuration	Trainable parameters	Training time (s)	Inference time per node (ms)
CiteSeer	Deep Sets	–	981 894	132.6 ± 1.0	0.596 ± 0.029
	Set Transformer	SAB	1 264 262	567.3 ± 4.0	1.185 ± 0.004
		ISAB, $m_{\text{ind}} = 8$	1 465 478	969.2 ± 9.9	2.230 ± 0.026
		ISAB, $m_{\text{ind}} = 16$	1 467 526	1009.0 ± 32.8	2.303 ± 0.080
		ISAB, $m_{\text{ind}} = 32$	1 471 622	1014.6 ± 37.8	2.312 ± 0.080
	Janossy GRU	$P = 1$	1 064 454	222.3 ± 6.5	0.815 ± 0.012
		$P = 5$	1 064 454	285.1 ± 10.9	4.088 ± 0.176
		$P = 10$	1 064 454	349.7 ± 3.4	8.080 ± 0.112
		$P = 20$	1 064 454	517.2 ± 11.5	17.011 ± 0.266
	PointNet++	$(c_1, c_2) = (4, 2)$	1 149 558	454.9 ± 5.7	1.710 ± 0.035
		$(c_1, c_2) = (8, 4)$	1 149 558	461.4 ± 3.6	1.747 ± 0.035
		$(c_1, c_2) = (16, 8)$	1 149 558	489.9 ± 16.1	1.851 ± 0.061
Synthetic	Deep Sets	–	35 458	58.7 ± 0.4	0.180 ± 0.003
	Set Transformer	SAB	317 826	517.9 ± 2.5	1.006 ± 0.010
		ISAB, $m_{\text{ind}} = 8$	519 042	903.2 ± 2.2	1.976 ± 0.016
		ISAB, $m_{\text{ind}} = 16$	521 090	909.7 ± 4.6	1.990 ± 0.026
		ISAB, $m_{\text{ind}} = 32$	525 186	909.2 ± 5.0	1.965 ± 0.018
	Janossy GRU	$P = 1$	118 018	236.7 ± 1.7	0.803 ± 0.011
		$P = 5$	118 018	289.1 ± 0.8	3.883 ± 0.028
		$P = 10$	118 018	356.1 ± 3.4	7.949 ± 0.155
		$P = 20$	118 018	485.9 ± 0.7	15.111 ± 0.029
	PointNet++	$(c_1, c_2) = (4, 2)$	144 002	345.7 ± 0.8	1.080 ± 0.002
		$(c_1, c_2) = (8, 4)$	144 002	389.1 ± 3.5	1.314 ± 0.019
		$(c_1, c_2) = (16, 8)$	144 002	424.6 ± 2.4	1.549 ± 0.006

The training times of Janossy GRU differ only slightly between the two datasets, despite the substantially higher parameter count on CiteSeer. In contrast, for Deep Sets, PointNet++ and the Set Transformer, the values measured on CiteSeer are predominantly higher than those on the synthetic dataset. The standard deviations are generally small in relation to the respective mean values. The observed order of the configurations is therefore largely stable across the five seeds.

There are clear differences between the configurations in terms of inference times per node. Deep Sets achieves the lowest values, at 0.596 ± 0.029 milliseconds on CiteSeer and 0.180 ± 0.003 milliseconds on the synthetic dataset. Janossy GRU, with $P = 1$ and an inference time of approximately 0.8 milliseconds per node, is also among the faster configurations. However, as the number of inference permutations increases, the time required rises approximately in proportion to this increase. For $P = 20$, 17.011 ± 0.266 milliseconds are required on CiteSeer and 15.111 ± 0.029 milliseconds on the synthetic dataset. Compared with $P = 1$, this corresponds to approximately twenty times the inference time. Janossy GRU with $P = 20$ thus exhibits the highest inference time of all the configurations examined, despite having fewer parameters than PointNet++ and the Set Transformer. Depending on the configuration, PointNet++’s inference times range between 1.080 and 1.851 milliseconds per node. On both datasets, these times increase with the number of centroid points used. The SAB variant takes 1.185 ± 0.004 milliseconds on CiteSeer and 1.006 ± 0.010 milliseconds on the synthetic dataset. The ISAB variants require approximately 2.0 to 2.3 milliseconds per node. Increasing the number of inducing points from 8 to 32 results in only minor changes to the inference time.

Overall, Deep Sets combines competitive classification results with the lowest parameter count and the shortest training and inference times. The additional computational effort of the more complex configurations is not accompanied by consistently higher classification performance in the present experiments.

5 Discussion

Before interpreting the individual datasets, their differing characteristics must be taken into account. The synthetic dataset presents a roughly balanced binary classification problem with a fixed neighbourhood size of $k = 10$. CiteSeer, by contrast, comprises six classes and exhibits varying node degrees. Furthermore, the synthetic features were generated in a Euclidean space, whilst CiteSeer contains high-dimensional document features. The datasets also have different input and output dimensions, which means that the number of parameters in the models also differs. The absolute classification scores, model sizes and runtime of the two datasets are therefore only directly comparable to a limited extent.

5.1 Synthetic dataset

On the synthetic dataset, the classification results of the models under investigation are very close to one another. The differences between the best configurations are smaller than their respective standard deviations, meaning that no clear superiority can be inferred from the mean values. The similar values of accuracy, Macro-F1 and Weighted-F1 suggest that neither class dominates the overall evaluation. Nevertheless, the pattern of precision and recall

indicates that the models predict class 0 more frequently and assign class 1 more cautiously. As this pattern occurs across all architectures, it may be related more to the dataset construction than to any individual model. The more pronounced nature and greater variation observed in PointNet++ could additionally indicate greater sensitivity in the hierarchical sampling and max pooling. As all architectures consider the features of the current node separately and use a comparable classification function, they differ in particular in their processing of the neighbourhood. The various neighbourhood encoders appear to generate similarly informative representations for this classification problem, but exhibit substantially different computational costs.

Deep Sets achieves a competitive result with a test accuracy of 0.770 ± 0.025 . The gap to the best Janossy configuration is merely 0.008, which is small in relation to the observed variance. One possible explanation lies in the construction of the synthetic target values. The label modification depends on the class composition of the neighbourhood and therefore on an aggregated property of its elements. Since the node features are sampled from class-conditional Gaussian distributions, the sum of the transformed neighbour features may allow Deep Sets to approximate this local composition relatively well. Direct interactions between individual neighbours would then be of only limited additional benefit for this task. At the same time, with 35 458 parameters, Deep Sets has the smallest model size and the shortest training and inference times. The model offers a particularly favourable starting point on this dataset when predictive performance and computational cost are considered together.

Set Transformer offers no advantage over the simpler architectures, despite its ability to model direct interactions between neighbours. The SAB variant achieves a test accuracy of 0.767 ± 0.038 , whilst ISAB with 8 inducing points yields 0.754 ± 0.023 , the lowest result of all configurations. Increasing the number of inducing points to 16 or 32 improves performance compared with 8, but does not result in any further difference between these two larger configurations. The limited benefit of the attention blocks could be related to the fixed neighbourhood size of ten elements. With such small sets, there are comparatively few interactions available for which self-attention might offer an advantage over simple aggregation. Nor does the efficiency advantage of ISAB, as explained in the theory section, come into play under these conditions. This advantage specifically requires that the number of inducing points be substantially smaller than the input set. With ten neighbours, this condition is only partially met, if at all. Furthermore, each ISAB contains two MABs, whilst in the present implementation each neighbourhood is processed individually with a batch size of one. This architecture results in higher practical computational costs. Accordingly, ISAB requires approximately 903 to 910 seconds of training time on the synthetic dataset and thus considerably more than SAB or the other model families. Under the conditions investigated, the Set Transformer therefore does not achieve a favourable balance between classification performance and computational cost.

Janossy GRU achieves the highest mean test accuracy of 0.778 ± 0.025 with $P = 20$. By averaging the predictions across several random neighbourhood permutations, Janossy GRU reduces the dependence of the sequence-sensitive GRU on a single order. A larger P thus improves the approximation of a permutation-invariant prediction, but at the same time increases the computational cost. Specifically, whilst the accuracy does increase gradually with P , the improvement between configurations remains small. At the same time, the inference time increases from

0.803 ± 0.011 milliseconds for $P = 1$ to 15.111 ± 0.029 milliseconds for $P = 20$. The additional improvement remains small and does not increase proportionally with the almost linear growth in inference time. Taking this relationship into account, the use of $P = 20$ cannot be unequivocally favoured, despite its highest mean value.

PointNet++ achieves a test accuracy of 0.775 ± 0.019 with $(c_1, c_2) = (16, 8)$ and is thus also close to the best Janossy configuration. As the number of centroids increases, the average performance improves slightly. One possible explanation is that larger sets of centroids preserve more local regions of the neighbourhood across the two levels of abstraction. With a fixed neighbourhood size of ten elements, this may reduce the loss of information caused by hierarchical sampling. Furthermore, the synthetic dataset was generated in a Euclidean feature space. The FPS and kNN operations used by PointNet++ may capture a local structure in this space that is suitable for classification. Whether the Euclidean structure is responsible for this result cannot be determined from the present experiments directly. However, the slight performance improvement is accompanied by higher training and inference times.

Overall, the more complex neighbourhood encoders show only minor improvements over Deep Sets on the synthetic dataset. Janossy Pooling achieves the highest mean score, whilst PointNet++ benefits slightly from larger centroid sets. However, both advantages remain small and come at the cost of longer running times. Deep Sets achieves almost the same classification performance with substantially lower computational effort. The results therefore suggest that a simple permutation-invariant aggregation is already largely sufficient for the structure of this synthetic classification problem.

5.2 CiteSeer dataset

On CiteSeer, there are more pronounced differences between the model families than on the synthetic dataset. The gap between Macro-F1 and Weighted-F1 indicates uneven classification performance across the six classes. This is confirmed by the class-specific results in Table 3. In particular, the low recall for class 0 occurs across all model families and therefore suggests a class- or dataset-specific difficulty rather than a problem with a single architecture. Possible causes could include the class distribution, less distinctive features, or specific structural properties of the associated nodes. It is not possible to clearly distinguish between these explanations on the basis of the available results.

The neighbourhood structure distinguishes CiteSeer clearly from the synthetic dataset. Almost two-thirds of the CiteSeer test nodes have only one or two neighbours. Isolated nodes achieve the lowest accuracy across all models. As their neighbourhood is represented by a zero vector in the shared implementation, their prediction depends solely on the features of the current node. The initial observed increase in accuracy with node degree is consistent with the fact that additional neighbours can provide helpful context. Of particular relevance is the densely populated degree group 1–2, in which Deep Sets and Janossy GRU achieve higher accuracies than PointNet++ and the Set Transformer. This contributes substantially to the differences in overall performance. In contrast, the results for the higher degree groups are only comparable to a limited extent due to their smaller sample sizes and

greater uncertainty. Against this background, the results of the individual models can be categorised based on their respective neighbourhood processing.

Deep Sets also achieves performance on CiteSeer that is close to that of the best configurations and outperforms PointNet++ as well as most Set Transformer variants in terms of Macro-F1. Simple sum aggregation therefore appears to generate a suitable representation even with varying neighbourhood sizes. This could be particularly related to the predominantly small neighbourhoods. With only one or two neighbours, there are hardly any complex interactions that could be additionally modelled by more complex encoders. In these cases, a shared transformation followed by sum aggregation may already capture a large proportion of the available neighbourhood information. Combined with the smallest model size and the shortest runtime, Deep Sets therefore achieves a favourable balance between performance and computational cost on CiteSeer as well.

Set Transformer achieves similar test accuracies to PointNet++, but in some cases still exhibits lower macro-F1 scores. Here, too, the small neighbourhoods may limit the architecture’s usefulness. Self-attention is trivial for a neighbourhood with only one element and, with two elements, can capture only a very limited number of interactions. Consequently, the additional expressiveness of the attention blocks can hardly be utilised for a large proportion of the nodes. For the ISAB variants, there is the additional factor that the number of inducing points exceeds the size of most neighbourhoods. The reduction in attention complexity intended for large neighbourhood sets therefore does not materialise. Instead, the two MABs in each ISAB, as well as the node-by-node processing, incur additional overhead. This offers a plausible explanation for why, in practice, ISAB is slower than SAB without achieving a consistent improvement in classification performance.

Janossy GRU achieves the highest test accuracy with $P = 10$, whilst $P = 20$ achieves the highest macro-F1. However, the difference between the two configurations is small. One possible explanation for the limited benefit of additional permutations also lies in the small neighbourhoods. For a neighbourhood with one element, there is only one possible order, whilst for two elements there are merely two different permutations. A high number of repeated evaluations can hardly reduce the dependence on order any further in these cases. Nevertheless, averaging multiple permutations could contribute to the comparatively low variance and the balanced class-specific results observed for $P = 20$. However, as the inference time roughly doubles compared to $P = 10$, the additional effort is not offset by a clear performance benefit.

PointNet++ performs substantially worse on CiteSeer than on the synthetic dataset. Both the neighbourhood sizes and the metric used are possible explanations for this. If a neighbourhood contains fewer elements than the requested number of centroids, the implementation limits the number of centroids to the available elements. For the majority of CiteSeer nodes, the different centroid configurations can therefore produce little or no actual hierarchical reduction. This could explain why larger sets of centroids do not result in a consistent improvement on CiteSeer. Furthermore, FPS and kNN are performed using Euclidean distances between the original high-dimensional document features. These distances do not necessarily reflect a local structure that is as meaningful as that found in

the synthetic dataset, which was specifically generated in a Euclidean feature space. The learnt local coordinates cannot compensate for this problem prior to clustering, as they are only used for the relative encoding of the groups that have already been selected. Under these conditions, the additional effort involved in hierarchical abstraction therefore does not lead to higher overall performance.

Overall, Deep Sets and Janossy GRU provide the strongest classification results on CiteSeer. Janossy GRU achieves slightly higher classification scores, whilst Deep Sets requires substantially less computational effort. Under the investigated configurations, PointNet++ and the Set Transformer do not consistently outperform the simpler models. Their additional mechanisms therefore provide no clear advantage for the predominantly small neighbourhoods and high-dimensional document features of CiteSeer.

5.3 Practical implications and limitations

In terms of practical model selection, the experiments show that Deep Sets offers a particularly favourable trade-off between classification performance and computational cost under the conditions investigated. Janossy Pooling achieves slightly higher mean scores in some cases, but incurs significantly higher costs for many inference permutations. By contrast, the additional mechanisms of PointNet++ and the Set Transformer do not consistently lead to better predictive performance. These results relate exclusively to the datasets and configurations examined and do not allow for a general ranking of the model families.

A key limitation of the interpretation concerns the synthetic dataset, the properties of which depend heavily on the chosen generation parameters. The selection was made on the basis of a reference classifier, the target distribution and the proportion of modified labels. As a result, it was only possible to achieve a classification difficulty that was approximately comparable to that of CiteSeer. The tasks also differ in terms of the number of classes, feature structure and neighbourhood sizes. Furthermore, the final model evaluation is based on a single synthetic realisation (Seed 42), which limits its generalisability to other realisations.

The configuration space under investigation was also restricted. To ensure better comparability, factors such as the internal feature dimension, the learning rate and the maximum number of training epochs were standardised, whilst only selected architecture-specific parameters were varied. Furthermore, the models were deliberately implemented in a compact manner. For example, Janossy Pooling uses only one GRU layer, the Set Transformer uses two encoder blocks and PointNet++ uses two local set abstraction levels. No comprehensive hyperparameter optimisation or investigation of deeper variants was carried out. The results therefore describe the specific implementations and not the full potential of the underlying architectures.

The runtime measurements also depend on the implementation and the hardware used. In particular, the neighbourhoods are processed individually, which means that the potential for parallel computation is only utilised to a limited extent. The measured times are therefore suitable for comparing the implementations under consideration, but should not be interpreted as general statements about the efficiency of the model families. Furthermore, the

analysis is based on five seeds and does not include any statistical significance tests. Minor differences with overlapping standard deviations therefore do not allow for clear conclusions regarding the superiority of any particular model. The results for the very small node degree groups on CiteSeer should also be interpreted with particular caution due to their small sample sizes.

Further research should incorporate additional datasets with larger, more variable and more interaction-dependent neighbourhoods. It is precisely such tasks that could demonstrate under which conditions sequential processing, self-attention or hierarchical local abstraction actually offer advantages over simple aggregation. In addition, more extensive hyperparameter studies, ablations of individual architectural components, alternative distance metrics for PointNet++ and more vectorised implementations would be useful.

Overall, the experiments show that, under the conditions investigated, greater architectural complexity does not automatically lead to better classification performance. A simple permutation-invariant aggregation method was competitive and particularly efficient on both datasets. The advantages of more complex methods are only evident in isolated cases, such as for PointNet++ on the synthetic dataset or for Janossy Pooling in terms of average classification performance. Whether these architectures offer clear advantages with larger and structurally more complex neighbourhoods requires further investigation and more experiments.

6 Tool usage reflection

ChatGPT [6] was used during implementation to assist with debugging and improving computational efficiency. The queries submitted focused in particular on the correct use of GPU acceleration with CUDA. This included placing models, input data and newly generated tensors on the same device, as well as synchronising asynchronously executed CUDA operations during runtime measurements. The adopted suggestions mainly concerned the shared functions for device selection, CUDA synchronisation, training and evaluation in `deepset.py`. Minor adjustments to device management were also made in `janossy_gru.py`, `set_transformer.py` and `pointnet_plus.py`. No complete model implementation generated by ChatGPT was adopted. Instead, individual suggestions were incorporated into the existing implementations and adapted to their structure.

ChatGPT and DeepL [2] were used to provide linguistic support during the drafting of this report. As neither of the authors is a native English speaker, DeepL was used in particular to check spelling, grammar and sentence structure. ChatGPT also helped to identify unclear wording, repetitions and excessively detailed passages. The theoretical foundations and the interpretation of the experimental results were developed independently by the authors.

All changes involving ChatGPT or DeepL were reviewed by both authors. Suggestions regarding the code were compared with the existing implementation and tested as part of the experiments. Linguistic suggestions were checked against the source code, the results and the literature used. Suggestions that were inaccurate, too general or inappropriate in terms of content were revised or rejected. No AI-generated code or text passages were included

without being read, critically reviewed and, where necessary, revised.

Using ChatGPT improved our understanding of device-aware programming with PyTorch. In particular, we learned more about the consistent placement of tensors and the synchronisation required for reliable CUDA runtime measurements. The linguistic revision also improved our ability to identify overly dense formulations, overly general claims and unnecessary repetition. It thereby helped us determine an appropriate level of detail for a technically complete report.

7 References

- [1] Babak Alipanahi et al. “Predicting the Sequence Specificities of DNA- and RNA-Binding Proteins by Deep Learning”. In: *Nature Biotechnology* 33.(8) (2015), pp. 831–838. ISSN: 1546-1696. DOI: 10.1038/nbt.3300. URL: <https://doi.org/10.1038/nbt.3300>.
- [2] DeepL SE. *DeepL Translator*. <https://www.deepl.com/translator>. Machine translation and linguistic revision tool, Last accessed 1 September 2026. 2026.
- [3] Juho Lee et al. *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks*. 2019. arXiv: 1810.00825 [cs.LG]. URL: <https://arxiv.org/abs/1810.00825>.
- [4] Maxwell W. Libbrecht and William Stafford Noble. “Machine Learning Applications in Genetics and Genomics”. In: *Nature Reviews Genetics* 16.(6) (2015), pp. 321–332. ISSN: 1471-0064. DOI: 10.1038/nrg3920. URL: <https://doi.org/10.1038/nrg3920>.
- [5] Ryan L. Murphy et al. *Janossy Pooling: Learning Deep Permutation-Invariant Functions for Variable-Size Inputs*. 2019. arXiv: 1811.01900 [cs.LG]. URL: <https://arxiv.org/abs/1811.01900>.
- [6] OpenAI. *ChatGPT*. <https://chatgpt.com/>. Large language model used for debugging, code revision, and linguistic support, Last accessed 1 September 2026. 2026.
- [7] Charles R. Qi et al. *PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space*. 2017. arXiv: 1706.02413 [cs.CV]. URL: <https://arxiv.org/abs/1706.02413>.
- [8] Manzil Zaheer et al. *Deep Sets*. 2018. arXiv: 1703.06114 [cs.LG]. URL: <https://arxiv.org/abs/1703.06114>.